

Star Classification

Clelia Meloni, Anna Ciapparelli, Tommaso Roncaglio

2025-04-03

1 Business Understanding

The project focuses on the analysis of a dataset for the classification of astronomical objects, specifically stars, galaxies, and quasars. The classification is based on the main spectral characteristics of the objects, with data collected by the SDSS (Sloan Digital Sky Survey).

The main objective of the analysis is to create a predictive model capable of classifying detected astrophysical objects using specific spectral features. In addition, the project aims to achieve further benefits, including:

- Greater operational efficiency, thanks to reduced analysis times and lower costs associated with managing large datasets through the computerization of the process.
- Improved analysis of astronomical data, enabling more accurate classification.
- Greater efficiency in scientific discoveries, thanks to the possibility of identifying new astronomical objects with greater precision.

To achieve these objectives, the original dataset is divided into three subsets with different proportions:

- **Training set:** used to train the model.
- **Validation set:** useful for optimizing parameters and preventing overfitting.
- **Test set:** used to evaluate model performance.

The analysis involves comparing several classification models, including:

- **Discriminant Analysis:**
 - **LDA (Linear Discriminant Analysis):** assumes that the classes have normal distributions with the same covariance matrix, leading to linear decision boundaries.
 - **QDA (Quadratic Discriminant Analysis):** allows a different covariance matrix for each class, generating quadratic decision boundaries.
- **K-Nearest Neighbors (KNN):** a method based on the proximity of points in the feature space.
- **Decision Trees:**
 - They iteratively split the dataset according to the values of the predictor variables.
 - They are interpretable and do not require the transformation of categorical variables.
 - They can suffer from overfitting if not properly pruned.

-
- Advanced versions such as Random Forests or Gradient Boosting Trees can improve performance by reducing variance.

The objective is to identify the model that best handles the data, balancing the bias-variance trade-off, avoiding misspecification and overfitting issues, and ensuring strong generalization ability.

The key questions guiding the analysis are:

1. Which variables have the greatest influence on classification?
2. How well can the model interpret the phenomenon under study?
3. Are there outliers common to all variables that could influence classification?
4. Are there underrepresented classes that could lead to the accuracy paradox?

2 Data Understanding

The dataset under analysis contains **100,000 observations** collected by the **Sloan Digital Sky Survey (SDSS)**, described through **17 numeric variables**. The last variable in the dataset is 'class', which is used to classify the observations into three categories: *Galaxy*, *Star*, *Quasar*.

ID variables

- `obj_ID` : Unique identifier of the object in the catalog.
- `spec_obj_ID` : Unique identifier for spectroscopic objects.
- `run_ID` : Identifies the specific scan.
- `rereun_ID` : Indicates how the image was processed.
- `cam_col` : Camera column used in the scan.
- `field_ID` : Field number used to identify the observed area.
- `plate` : ID of the plate that captured the spectrum.
- `MJD` : Modified Julian Date, indicating the time of the observation.
- `fiber_ID` : Identifies the optical fiber used to capture the light of the object.

These ID variables are different identifiers for the techniques and instruments used to process and scan the objects, such as plates, optical fibers, or the date on which the observation was carried out.

Variables for spatial coordinates

- `delta` : Declination (DEC) in degrees.
- `alpha` : Right ascension (RA) in degrees.

Celestial coordinates are used to identify the position of celestial bodies on the celestial sphere. They use the celestial equator, hour circles (or meridians), and the first point of Aries (γ) as references, namely the point where the Sun moves from the southern celestial hemisphere to the northern one. As additional information, polar distance (p) is sometimes used instead of declination; it is the angular distance of the celestial body from the north celestial pole and ranges from 0 degrees to 180 degrees (as complementary angles, $p + \delta = 90$ degrees).

Variables for photometric filters

- `u` : Near-ultraviolet filter (355 nm).
- `g` : Green filter (477 nm).
- `r` : Red filter (622 nm).
- `i` : Near-infrared filter (763 nm).
- `z` : Infrared filter (893 nm).

A star emits a flux of electromagnetic radiation that depends on its surface temperature. This radiation is perceived by the human eye as visible light, which determines the color of the star as it appears to us. However, the light emitted by a star has a continuous spectrum that extends over a wide range of frequencies, and its analysis makes it possible to understand how the celestial body behaves. To isolate light radiation in the different bands, photometric filters are used; these are carefully calibrated filters so that the percentage of luminous flux recorded at each frequency conforms to a predefined standard.

There are many photometric systems. The one considered in this dataset is the SDSS system, which includes the five optical bands described above.

Shift variable

- `redshift` : Phenomenon whereby the light emitted by receding objects has a longer wavelength than it had at emission.

The English term 'redshift' comes from the fact that the light moves toward the red end of the visible spectrum. The opposite case is 'blueshift', where the wavelength decreases when the light moves toward an observer.

The observation technique used in this dataset is photometry, which is simpler but provides a rather approximate estimate of redshift. The optimal alternative would be to estimate redshift using spectroscopy, but that technique is more complex because it is negatively affected by factors such as lack of light and non-professional instruments. In general, spectroscopy is the most accurate method for measuring redshift, but when it cannot be used, photometry provides a valid estimate.

Target variable

- `class` : Indicates the category of the object (*star*, *galaxy*, *quasar*).

We provide a brief description of these three categories to better clarify their meaning:

Agalaxy is a gravitationally bound system of stars and other components of matter.

Astar is a celestial body formed by plasma and gas at high temperature.

Aquasar is an extragalactic object that emits an enormous amount of energy.

Dataset presentation

```

library(MASS)
library(class)
library(rpart)
library(rpart.plot)
library(caret)
library(tidyverse)
library(gridExtra)
library(ROCR)

library(readxl)
star_classification <- read_excel("star_classification.xlsx")

for (i in 1:12) {
  star_classification[, i] <-
as.numeric(unlist(star_classification[, i]))
}
for (i in 13:14) {
  star_classification[, i] <-
as.character(unlist(star_classification[, i]))
}
for (i in 15:18) {
  star_classification[, i] <-
as.numeric(unlist(star_classification[, i]))
}

#View(star_class)
#str(star_classification)
star_classification$cam_col <-
as.factor(star_classification$cam_col)
set.seed(123)

train_size <- floor(0.80 * nrow(star_classification))
train_index <- sample(seq_len(nrow(star_classification)), size =
train_size)

training <- star_classification[train_index, ]
test <- star_classification[-train_index, ]

subtrain_size <- floor(0.65 * nrow(training))
subtrain_index <- sample(seq_len(nrow(training)), size =
subtrain_size)

sub_training <- training[subtrain_index, ]
validation <- training[-subtrain_index, ]
data <- sub_training
dim(data) # 52000    18

```

```
## [1] 52000    18
```

```
dim(validation) #28000    18
```

```
## [1] 28000    18
```

```
dim(test) #20000    18
```

```
## [1] 20000    18
```

```
# If needed, you can also check the first rows to make sure the  
data are formatted correctly:  
head(data)
```

```
# Handling ID variables  
length(unique(sub_training$obj_ID))
```

```
## [1] 46027
```

```
# 80,846 observations are unique, while the remaining 19,154 are  
repeated observations
```

In the dataset, each observation is unique (spec_obj_ID), but some of the objects observed in these observations appear more than once (obj_ID) and are therefore not singular. For this reason, we considered it appropriate to remove repeated observations from the dataset.

```
data <- subset(sub_training, !duplicated(obj_ID))  
  
validation <- subset(validation, !duplicated(obj_ID))  
  
test <- subset(test, !duplicated(obj_ID))
```

The dataset under analysis is divided into three subsets: a training set, used in the first phase to train the models and understand the relationships among the variables (80% of the total data); a validation set, used to estimate model parameters and evaluate the quality of the analyses (45% of the training set); and finally a test set (20% of the total data), which is used to evaluate the final performance of the selected model by observing its ability to generalize the analysis to new data. The decision to include a validation set in our project comes from the desire to limit the effects of underfitting or

overfitting that could arise from the dataset analysis. Through this split, we evaluate a series of classification models: K-Nearest Neighbors (KNN), multinomial regression, decision trees for classification, Linear Discriminant Analysis, and Quadratic Discriminant Analysis, with the objective of identifying the method that best classifies the data available to us.

```
# Removal of non-useful ID columns
data <- subset(data,select=-c(obj_ID, spec_obj_ID, plate,
fiber_ID, MJD, rerun_ID, run_ID, field_ID))
validation <- subset(validation,select=-c(obj_ID, spec_obj_ID,
plate, fiber_ID, MJD, rerun_ID, run_ID, field_ID))
test <- subset(test,select=-c(obj_ID, spec_obj_ID, plate,
fiber_ID, MJD, rerun_ID, run_ID, field_ID))
dim(validation);str(validation)
```

```
## [1] 26077    10
```

```
## tibble [26,077 × 10] (S3: tbl_df/tbl/data.frame)
## $ alpha      : num [1:26077] 198 212 212 215 312 ...
## $ delta      : num [1:26077] 63.9 11.3 65.8 48 -7.1 ...
## $ u          : num [1:26077] 24.4 22.3 23.4 24.8 20.2 ...
## $ g          : num [1:26077] 22.9 21 20.5 22.4 18.4 ...
## $ r          : num [1:26077] 21.1 21.1 18.9 20.6 17.6 ...
## $ i          : num [1:26077] 19.9 20.9 18.3 19.5 17.2 ...
## $ z          : num [1:26077] 19.4 20.5 17.8 19.2 17 ...
## $ cam_col    : Factor w/ 6 levels "1","2","3","4",...: 2 4 5 3 1
1 1 3 1 5 ...
## $ class      : chr [1:26077] "GALAXY" "QSO" "GALAXY"
"GALAXY" ...
## $ redshift: num [1:26077] 0.59466 2.41554 0.31397 0.55009
-0.00036 ...
```

```
dim(test);str(test)
```

```
## [1] 19018    10
```

```
## tibble [19,018 × 10] (S3: tbl_df/tbl/data.frame)
## $ alpha      : num [1:19018] 145 146 133 143 136 ...
## $ delta      : num [1:19018] 31.27 47.3 4.52 5.55 36.15 ...
## $ u          : num [1:19018] 24.8 21.7 21.3 26 20.7 ...
## $ g          : num [1:19018] 22.8 21.5 20.5 21.3 22.2 ...
## $ r          : num [1:19018] 22.6 21.3 18.4 19.6 24.8 ...
## $ i          : num [1:19018] 21.2 21.4 23.2 18.8 21.4 ...
## $ z          : num [1:19018] 21.6 21.2 18 18.3 22.8 ...
## $ cam_col    : Factor w/ 6 levels "1","2","3","4",...: 5 2 3 3 4
3 4 4 3 4 ...
## $ class      : chr [1:19018] "GALAXY" "QSO" "GALAXY"
"GALAXY" ...
## $ redshift   : num [1:19018] 0.779136 2.07568 0.250956 0.461278
0.000308 ...
```

```
dim(data);str(data)
```

```
## [1] 46027    10
```

```
## tibble [46,027 × 10] (S3: tbl_df/tbl/data.frame)
## $ alpha      : num [1:46027] 223 358 249 129 218 ...
## $ delta      : num [1:46027] 35.68 4.96 46.73 34.1 15.91 ...
## $ u          : num [1:46027] 22.4 23 25.2 23.7 24 ...
## $ g          : num [1:46027] 21 22.7 23.2 21.2 22.3 ...
## $ r          : num [1:46027] 19.2 20.9 21.8 19.3 20.7 ...
## $ i          : num [1:46027] 18.5 19.8 20.6 18.6 19.8 ...
## $ z          : num [1:46027] 18 19.3 19.7 18.2 19.1 ...
## $ cam_col    : Factor w/ 6 levels "1","2","3","4",...: 3 6 3 4 1
3 4 1 5 3 ...
## $ class      : chr [1:46027] "GALAXY" "GALAXY" "STAR"
"GALAXY" ...
## $ redshift   : num [1:46027] 0.360823 0.617715 0.000149 0.430309
0.000104 ...
```

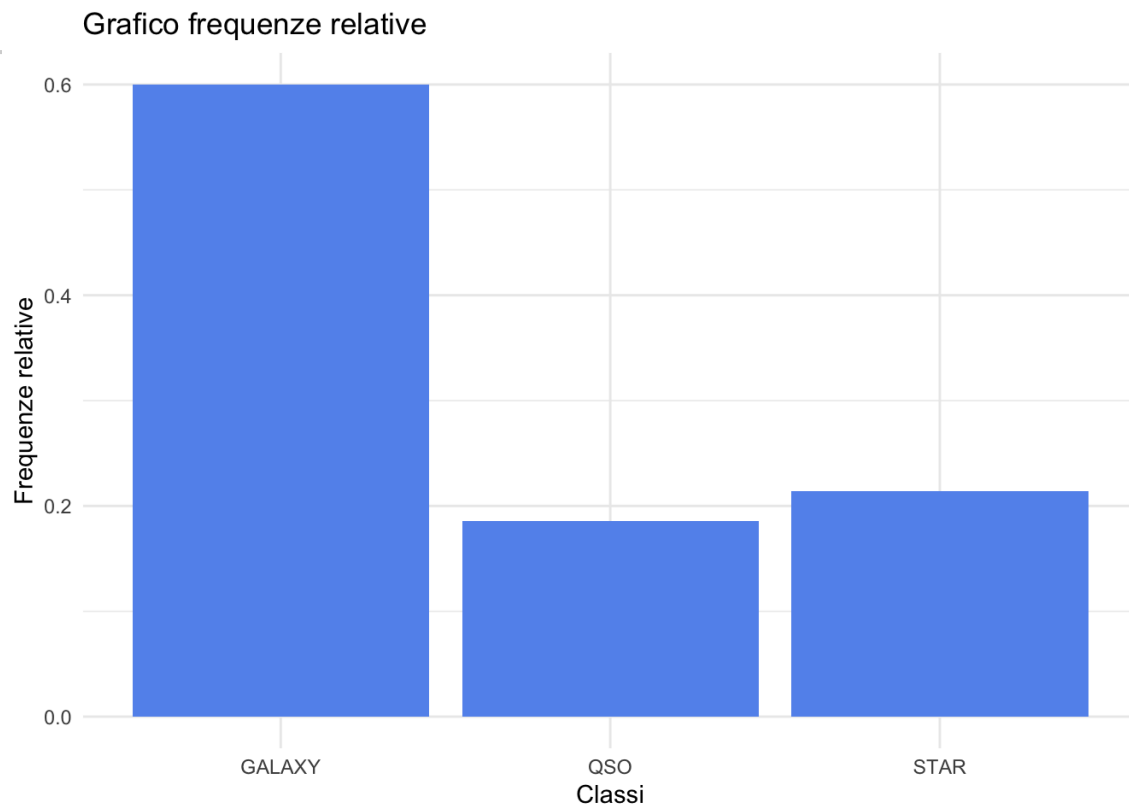
In addition to the object IDs mentioned above, there are other variables representing different identifiers for the techniques and instruments used to process and scan the objects. At the analysis level, these variables are not particularly significant and also take very different values over very wide ranges, making them complex to manage within the dataset. The only ID variable with a limited number of values is `cam_col`, which can be treated as a categorical variable with six levels and can therefore be handled relatively easily. For this reason, we decided to keep it in the final dataset and remove the others.

3Data Pre-Processing

```
# Exploratory analysis: relative frequency representation for
each class
summary(data)
```

```
##      alpha      delta      u
g      r      i
z      cam_col
## Min.   : 0.0055 Min.   :-17.636 Min.   : 12.26
Min.   : 10.51 Min.   : 10.07 Min.   : 10.96
Min.   : 10.23 1:6140
## 1st Qu.:128.4858 1st Qu.: 5.893 1st Qu.: 20.43 1st
Qu.: 19.07 1st Qu.: 18.23 1st Qu.: 17.82 1st Qu.:
17.54 2:7982
## Median :181.8082 Median : 23.576 Median : 22.30
Median : 21.19 Median : 20.19 Median : 19.44
Median : 19.03 3:8684
## Mean   :178.7627 Mean   : 24.207 Mean   : 216.82
Mean   : 206.26 Mean   : 194.91 Mean   : 205.21
Mean   : 180.82 4:9169
## 3rd Qu.:234.7692 3rd Qu.: 39.679 3rd Qu.: 23.81 3rd
Qu.: 22.19 3rd Qu.: 21.08 3rd Qu.: 20.44 3rd Qu.:
19.93 5:8320
## Max.   :359.9998 Max.   : 83.001 Max.   :27626.00
Max.   :27036.00 Max.   :23908.00 Max.   :24942.00 Max.
:22948.00 6:5732
##      class      redshift
## Length:46027 Min.   : -0.0996
## Class :character 1st Qu.: 0.0566
## Mode :character Median : 0.4363
## Mean   : 0.7723
## 3rd Qu.: 0.7008
## Max.   :2673.0000
```

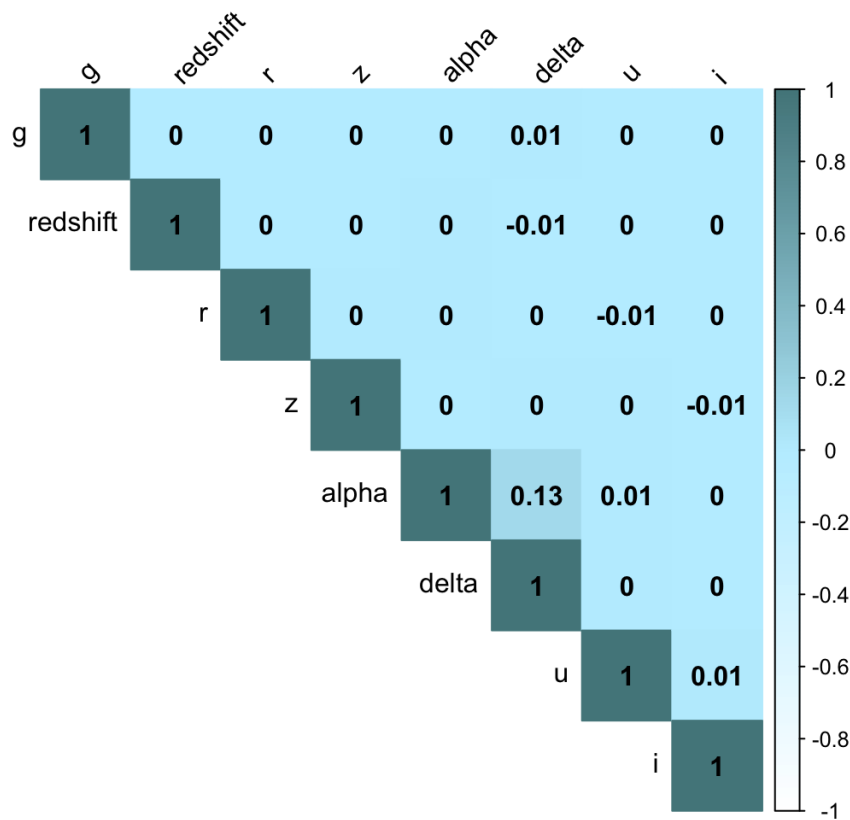
```
ggplot(data, aes(x = class)) + geom_bar(aes(y=after_stat(count)/
sum(after_stat(count))), fill = "cornflowerblue") +
  labs(title="Relative frequency chart", x="Classes", y="Relative
frequencies") + theme_minimal()
```

```
def <- data

# Conversion of classes into numeric values (0 = Galaxy, 1 =
Star, 2 = Quasar), useful for classification
def$class <- as.numeric(factor(data$class, levels = c("GALAXY",
"STAR", "QSO")))
def$class <- as.factor(data$class)

# Graphical visualization of class correlation
library(corrplot)
data_cor <- def[, -c(8,9)]
cor_matrix <- cor(data_cor)
corrplot(cor_matrix, method = "color",
          type = "upper",
          order = "hclust",
          addCoef.col = "black",
          col = colorRampPalette(c("white", "lightblue1",
"cadetblue4"))(200),
          tl.col = "black", tl.srt = 45)
```



For the purposes of the analysis, it is useful for the variables not to be correlated with each other, because this means that each variable contributes uniquely to the final dataset and that there is no redundancy in the information conveyed by the variables. We are therefore inclined to interpret this result positively and proceed with the analysis.

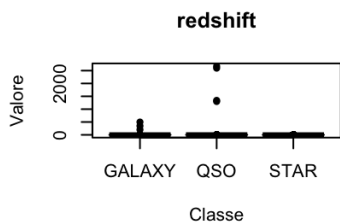
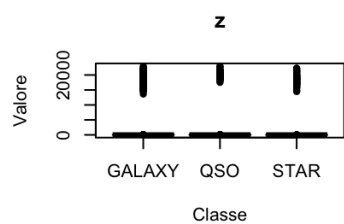
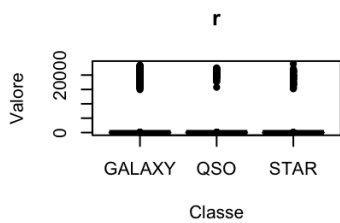
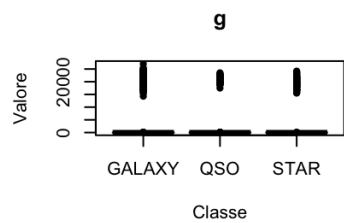
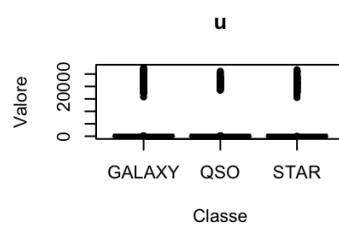
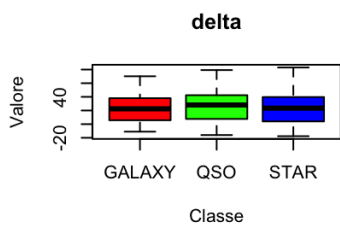
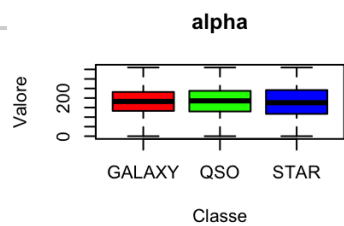
```
# Creation of boxplots for each numeric variable
numeric_vars <- def[ , sapply(def, is.numeric)]
num_vars <- ncol(numeric_vars)

par(mfrow = c(3, 3))

for (var in colnames(numeric_vars)) {
  boxplot(numeric_vars[[var]] ~ def$class,
    main = var,
    xlab = "Class",
    ylab = "Value",
    col = c("red", "green", "blue"),
    outline = TRUE,
    pch = 19)
}

par(mfrow=c(1,1))
```

—



```

# Outlier search and analysis
# Several outliers could influence the analysis, so we proceed
with their removal
# Selection of the columns of interest
#Training set
data_con_outliers <- def |> select(u, g, r, i, z)
#Validation set
validation_con_outliers <- validation |> select(u, g, r, i, z)

# Outlier removal using the interquartile range
#Training set
Q1 <- apply(data_con_outliers, 2, quantile, 0.25, na.rm = TRUE)
Q3 <- apply(data_con_outliers, 2, quantile, 0.75, na.rm = TRUE)
IQR <- Q3 - Q1
lower <- Q1 - 1.5 * IQR
upper <- Q3 + 1.5 * IQR
#Validation set
Q1_2 <- apply(validation_con_outliers, 2, quantile, 0.25, na.rm =
TRUE)
Q3_2 <- apply(validation_con_outliers, 2, quantile, 0.75, na.rm =
TRUE)
IQR_2 <- Q3 - Q1
lower_2 <- Q1 - 1.5 * IQR
upper_2 <- Q3 + 1.5 * IQR

# Rows to remove
#Training set
outlier_rows <- which(rowSums(data_con_outliers < lower |
data_con_outliers > upper, na.rm = TRUE) > 0)
#Validation set
outlier_rows2 <- which(rowSums(validation_con_outliers < lower_2
| validation_con_outliers > upper_2, na.rm = TRUE) > 0)

#Training set
data_clean <- def[-outlier_rows, ]
nrow(def) - nrow(data_clean) # 7,919 rows were removed

```

```
## [1] 7919
```

```

#Validation set
data_clean2 <- validation[-outlier_rows2, ]
nrow(validation) - nrow(data_clean2) # 3,994 rows were removed

```

```
## [1] 4360
```

From a previous analysis of the dataset based on the boxplots, we observed that the redshift variable has less pronounced outliers than the others. The approach used is a logarithmic transformation (we add 1 inside the logarithm to handle negative values; for redshift the minimum value was -0.09959, so adding 1 satisfies the mathematical conditions required to apply the logarithm). We include this transformation in our analysis because its distribution becomes more homogeneous and the outliers become easier to manage, as they are concentrated in the tails of the distribution.

```
data_clean$log_redshift <- log(data_clean$redshift+1)

data_clean2$log_redshift <- log(data_clean2$redshift+1)
```

With the objective of bringing all variables into a single variance range, we proceed with data normalization, limiting the value ranges to [0,1] without changing the overall distribution. This choice is motivated by the fact that an initial analysis of the dataset using the summary() command showed that the individual columns take values across wide ranges, which also differ substantially from one another.

```

# Normalization
#Training set
data_normalized <- data_clean
numeric_columns <- sapply(data_normalized, is.numeric)
data_normalized[numeric_columns] <-
lapply(data_normalized[numeric_columns],
        function(x) (x -
min(x)) / (max(x) - min(x)))

numeric_vars <- data_clean[sapply(data_clean, is.numeric)]
#Validation set
data_normalized2 <- data_clean2
numeric_columns2 <- sapply(data_normalized2, is.numeric)
data_normalized2[numeric_columns2] <-
lapply(data_normalized2[numeric_columns2],
        function(x) (x -
min(x)) / (max(x) - min(x)))

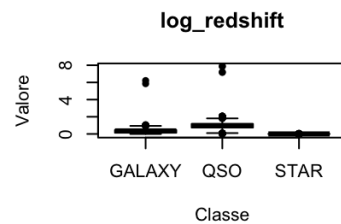
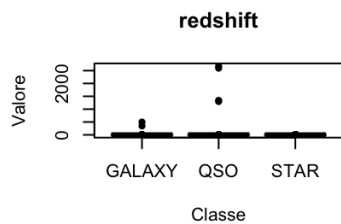
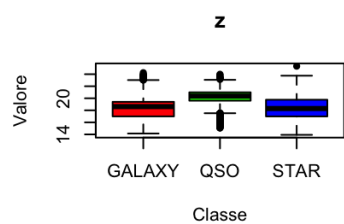
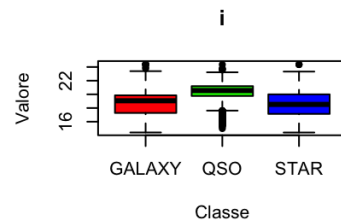
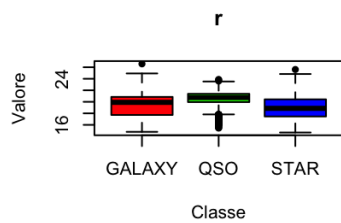
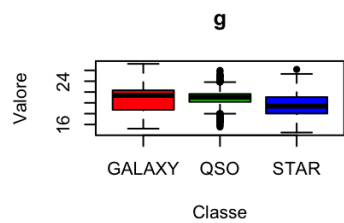
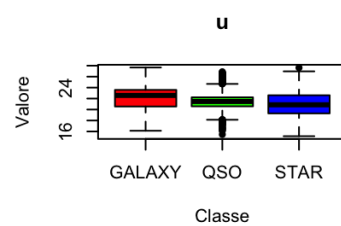
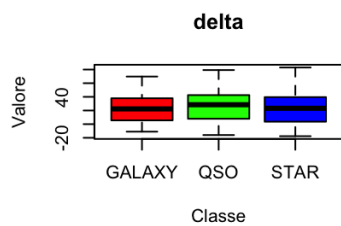
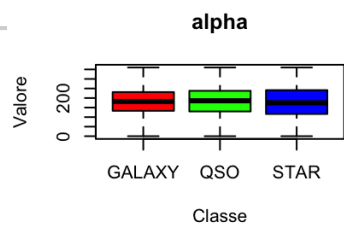
numeric_vars2 <- data_clean2[sapply(data_clean2, is.numeric)]

# Generate boxplots for each numeric variable by class after the
required data adjustments
par(mfrow = c(3, 3))

for (var in colnames(numeric_vars)) {
  boxplot(numeric_vars[[var]] ~ data_clean$class, # Use
data_clean$class for consistency
          main = var,
          xlab = "Class",
          ylab = "Value",
          col = c("red", "green", "blue"),
          outline = TRUE,
          pch = 19)
}

```

—



```

par(mfrow=c(1,1))

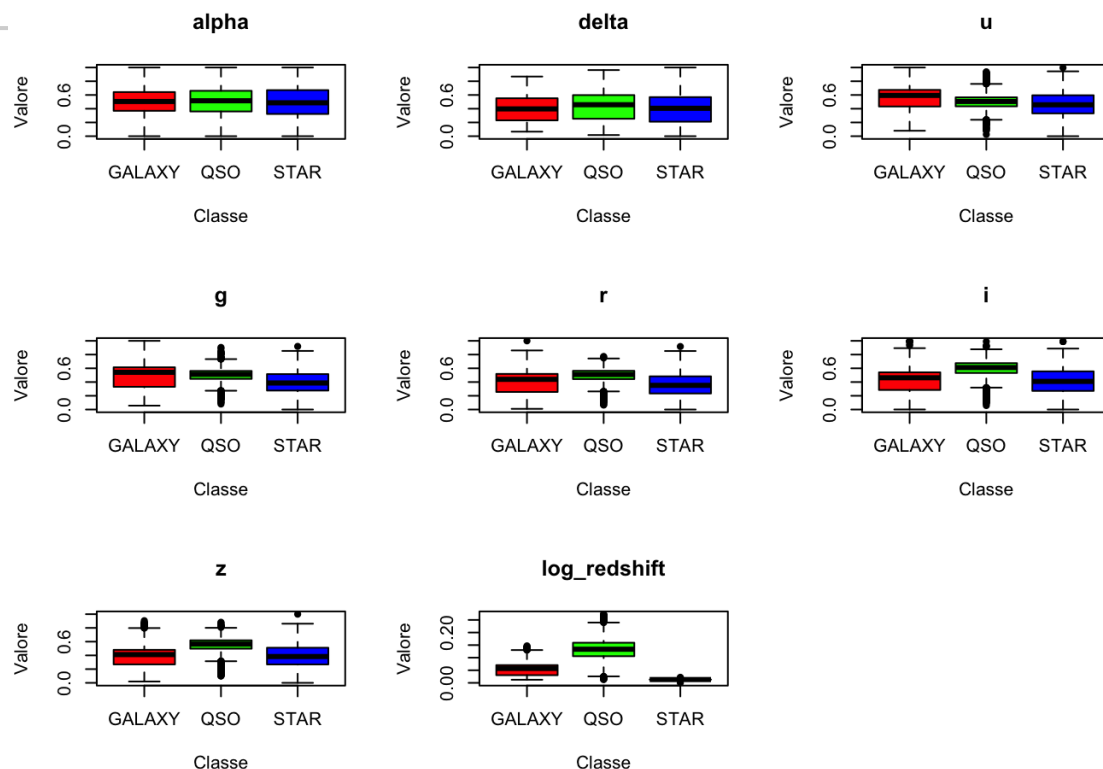
# We observe that the log_redshift variable contains outliers, so
# we set the threshold at 0.7
# and we remove values that differ significantly from the overall
# distribution,
# namely values that exceed this threshold because they are far
# beyond the upper whisker of the boxplot

#Training set
data_normalized <- data_normalized |>
  select(-redshift)|>
  filter(!(log_redshift > 0.7))
normalized_numeric_vars <-
data_normalized[sapply(data_normalized, is.numeric)]
#Validation set
data_normalized2 <- data_normalized2 |>
  select(-redshift)|>
  filter(!(log_redshift > 0.7))
normalized_numeric_vars2 <-
data_normalized2[sapply(data_normalized2, is.numeric)]

# We plot the training set to see how the distribution of
log_redshift changed
# after applying the threshold-based removal
par(mfrow = c(3, 3))

for (var in colnames(normalized_numeric_vars)) {
  boxplot(normalized_numeric_vars[[var]] ~
data_normalized$class,
          main = var,
          xlab = "Class",
          ylab = "Value",
          col = c("red", "green", "blue"),
          outline = TRUE,
          pch = 19)
}
par(mfrow = c(1,1))

```

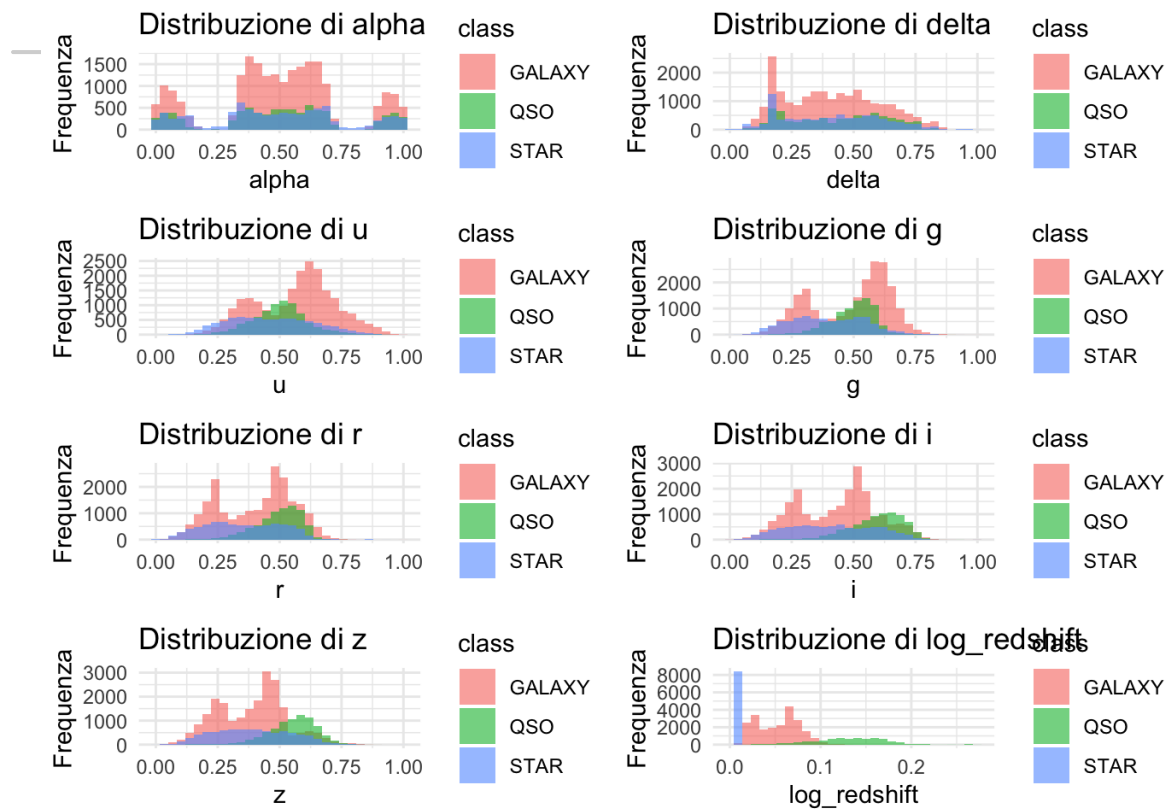
```
library(gridExtra)

# Graphical representation of variable distributions conditional
on class
histograms <- list()
normalized_numeric_vars <- colnames(data_normalized)
[sapply(data_normalized, is.numeric)]

for (i in 1:length(normalized_numeric_vars)) {
  var <- normalized_numeric_vars[i]

  # Create the plot
  histograms[[i]] <- ggplot(data_normalized, aes_string(x = var,
fill = "class")) +
    geom_histogram(bins = 30, alpha = 0.6, position = "identity")
+
  labs(title = paste("Distribution of", var), x = var, y =
"Frequency") +
  theme_minimal()
}

grid.arrange(histograms[[1]], histograms[[2]], histograms[[3]],
histograms[[4]], histograms[[5]],
             histograms[[6]], histograms[[7]], histograms[[8]],
             ncol = 2)
```



The graph shows the frequency distributions of the numeric variables under analysis, conditional on class. The variable that appears most discriminating is the transformed redshift, which is highly informative in distinguishing the classes. The variables alpha and delta show similar distributions, differing only in their frequency peaks. The broader and more pronounced distribution for the GALAXY class is understandable given the size of that class. The variables identifying the photometric magnitudes again show a distinction among classes, although it is less pronounced than in the distribution of the transformed log_redshift. Specifically, the GALAXY label shows a bimodal pattern, the QSO label shows an asymmetric distribution, and STAR shows a flatter distribution across these five variables.

```
# Class balancing
set.seed(123)
print(table(data_normalized$class))
```

```
##
## GALAXY    QSO    STAR
##  21902    7798    8402
```

```
# Undersampling of the most represented class
galaxy_class <- data_normalized %>%
  filter(class == 'GALAXY')

n0 <- nrow(galaxy_class)
set.seed(123)
to_remove <- galaxy_class %>%
  slice_sample(n = 10000)
dati <-<- data_normalized %>%
  anti_join(to_remove, by = names(data_normalized))

print(table(dati$class))
```

```
##
## GALAXY    QSO    STAR
##  11902    7798    8402
```

The GALAXY class is more represented than the others. For this reason, it was undersampled in order to make the distribution of observations across the three classes more homogeneous. Even after this operation, we are still in a context of imbalanced classes, because GALAXY remains more numerous than the others. Consequently, this must be taken into account when interpreting the results, considering the possibility of compromised evaluation metrics such as model accuracy.

4 Modeling and Evaluation

4.1 KNN (K-Nearest Neighbours)

```
set.seed(123)
library(MLmetrics)

numeric.training <- dati |> select(alpha, delta, u, g, r, i, z,
  log_redshift, cam_col)
numeric.validation <- data_normalized2 |> select(alpha, delta, u,
  g, r, i, z, log_redshift, cam_col)

class.validation <- data_normalized2$class
class.training <- dati$class

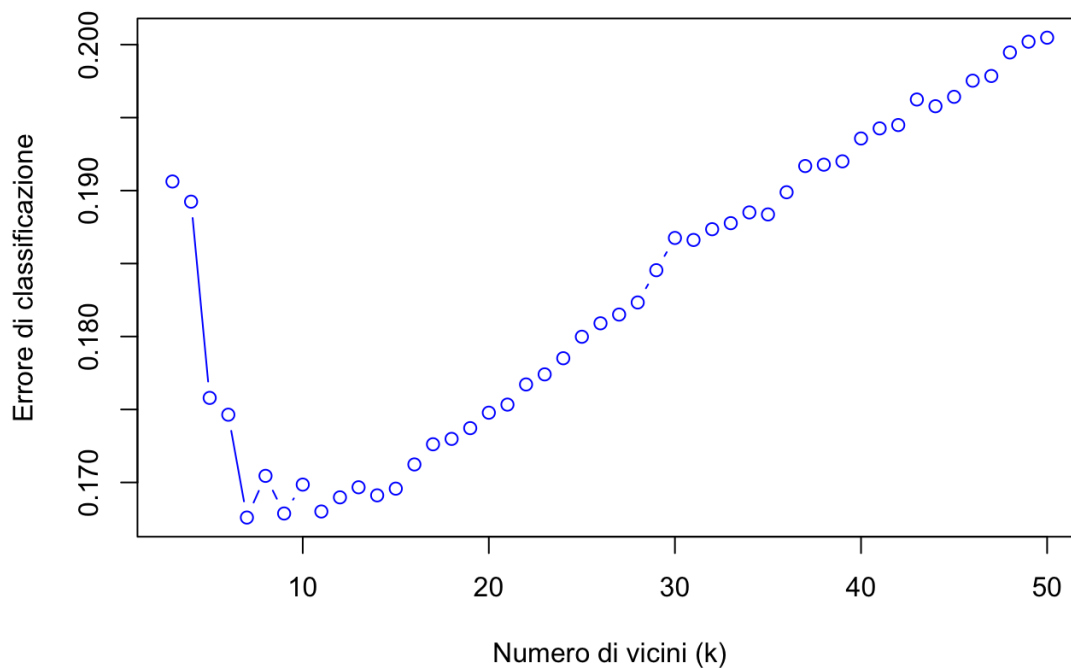
calc_class_err <- function(act, pred) {
  mean(act != pred)
}
k_to_try <- 3:50
err_k <- sapply(k_to_try, function(k) {
  pred <- knn(train = numeric.training,
    test = numeric.validation,
    cl = class.training,
    k = k)
  calc_class_err(class.validation, pred)
})

print(err_k)
```

```
## [1] 0.1906231 0.1892415 0.1757933 0.1746419 0.1675954
0.1704509 0.1678718 0.1698522 0.1680099 0.1689771 0.1696679
0.1691153 0.1695758 0.1712338 0.1726155 0.1729839 0.1737208
## [18] 0.1747801 0.1753327 0.1767144 0.1774052 0.1785106
0.1799843 0.1809054 0.1815042 0.1823332 0.1845438 0.1867545
0.1866163 0.1873532 0.1877677 0.1885046 0.1883664 0.1898862
## [35] 0.1916824 0.1917745 0.1920048 0.1935707 0.1942615
0.1944918 0.1962419 0.1957813 0.1964261 0.1975314 0.1978538
0.1994658 0.2002026 0.2004790
```

```
plot(k_to_try, err_k, type = 'b', col = 'blue',
  xlab = 'Number of neighbors (k)', ylab = 'Classification
error',
  main = 'Classification Error for Different Values of k')
```

Errore di Classificazione per Diversi Valori di k



```
best_k <- k_to_try[which.min(err_k)]  
best_k
```

```
## [1] 7
```

```
knn_pred <- knn(train = numeric.training,  
                test = numeric.validation,  
                cl = class.training,  
                k = best_k)  
  
knn_pred <- as.factor(knn_pred)  
class.validation <- factor(class.validation, levels =  
  levels(knn_pred))  
conf_matrix <- confusionMatrix(knn_pred, class.validation)  
print(conf_matrix) # Accuracy : 0.8323
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction GALAXY   QS0   STAR
##   GALAXY  11374   726  1543
##   QS0       203  3592   197
##   STAR       869   114  3095
##
## Overall Statistics
##
##           Accuracy : 0.8318
##           95% CI : (0.8268, 0.8368)
##   No Information Rate : 0.5732
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.6999
##
##   Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##           Class: GALAXY Class: QS0 Class: STAR
## Sensitivity           0.9139      0.8105      0.6401
## Specificity           0.7552      0.9769      0.9418
## Pos Pred Value        0.8337      0.8998      0.7590
## Neg Pred Value        0.8672      0.9526      0.9013
## Prevalence            0.5732      0.2041      0.2227
## Detection Rate        0.5238      0.1654      0.1425
## Detection Prevalence  0.6283      0.1839      0.1878
## Balanced Accuracy     0.8345      0.8937      0.7909
```

```
#F1SCORE
f1 <- F1_Score(y_pred = knn_pred, y_true = class.validation) ; f1
```

```
## [1] 0.8719384
```

Based on the results obtained for accuracy, which in the context of imbalanced classes was paired with the F1-score because it is more precise in this setting, and on the ROC curves for the individual classes, we consider the KNN method a valid option for classifying our data. It appears to produce good results in classification precision (F1-score = 0.87). We compare it with the other models to determine which is actually the best for this type of data.

Since the ROC curve refers to binary contexts, in this dataset analysis we adapt our classes to this type of evaluation by applying a one-vs-all strategy, so that each class is compared with the others. We can therefore treat our three-class classification problem as a binary problem, where each individual label is compared with the other two combined.

```
# ROC CURVE AND AUC
```

```
library(ROCR)
```

```
# Convert all classes to factors to ensure the factor type is  
respected
```

```
class.training <- factor(class.training)
```

```
class.validation <- factor(class.validation, levels =  
levels(class.training))
```

```
### "Galaxy" vs "non-Galaxy"
```

```
train_bin1 <- ifelse(class.training == "GALAXY", "1", "0")
```

```
valid_bin1 <- ifelse(class.validation == "GALAXY", 1, 0)
```

```
knn_bin1 <- knn(train = numeric.training,  
               test = numeric.validation,  
               cl = train_bin1,  
               k = best_k,  
               prob = TRUE)
```

```
prob1 <- attributes(knn_bin1)$prob
```

```
prob1 <- ifelse(knn_bin1 == "1", prob1, 1 - prob1)
```

```
pred1 <- prediction(prob1, valid_bin1)
```

```
perf1 <- performance(pred1, "tpr", "fpr")
```

```
auc1 <- performance(pred1, "auc")@y.values[[1]]
```

```
### "Quasar" vs "non-Quasar"
```

```
train_bin2 <- ifelse(class.training == "QS0", "1", "0")
```

```
valid_bin2 <- ifelse(class.validation == "QS0", 1, 0)
```

```
knn_bin2 <- knn(train = numeric.training,  
               test = numeric.validation,  
               cl = train_bin2,  
               k = best_k,  
               prob = TRUE)
```

```
prob2 <- attributes(knn_bin2)$prob
```

```
prob2 <- ifelse(knn_bin2 == "1", prob2, 1 - prob2)
```

```
pred2 <- prediction(prob2, valid_bin2)
```

```
perf2 <- performance(pred2, "tpr", "fpr")
```

```
auc2 <- performance(pred2, "auc")@y.values[[1]]
```

```
### "Star" vs "Non-Stars"
```

```
train_bin3 <- ifelse(class.training == "STAR", "1", "0")
```

```
valid_bin3 <- ifelse(class.validation == "STAR", 1, 0)
```

```
knn_bin3 <- knn(train = numeric.training,
```



```

test = numeric.validation,
cl = train_bin3,
k = best_k,
prob = TRUE)

prob3 <- attr(knn_bin3, "prob")
prob3 <- ifelse(knn_bin3 == "1", prob3, 1 - prob3)

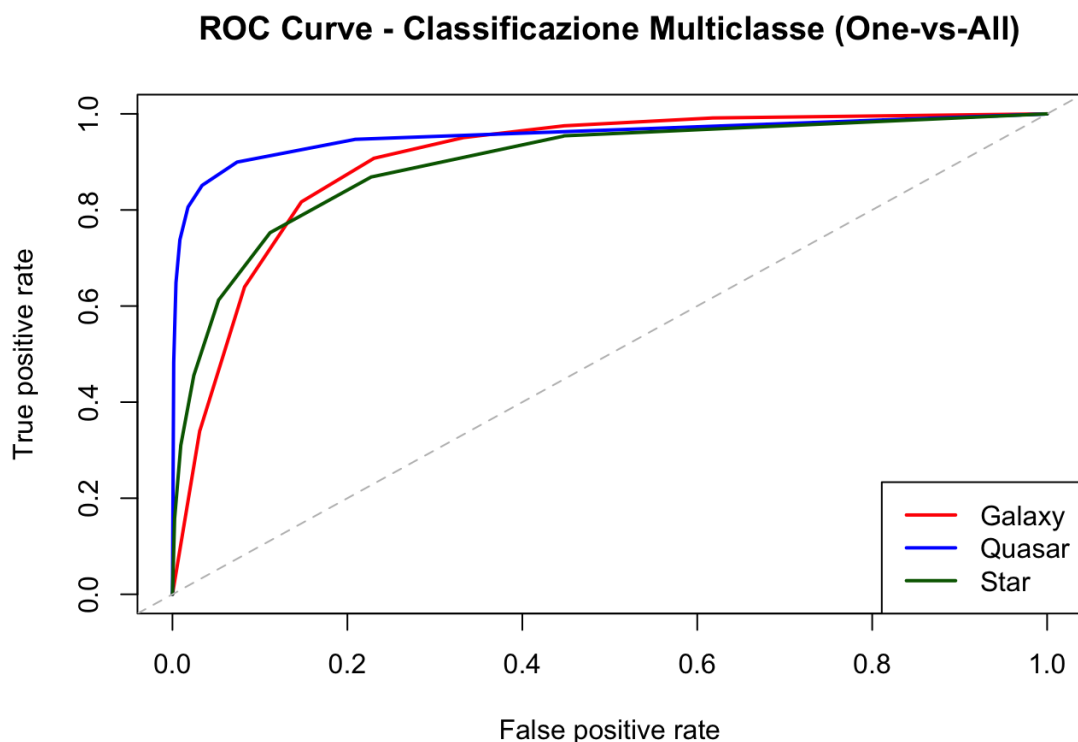
pred3 <- prediction(prob3, valid_bin3)
perf3 <- performance(pred3, "tpr", "fpr")
auc3 <- performance(pred3, "auc")@y.values[[1]]

### Graphical representation of the curve for the three classes
plot(perf1, col = "red", lwd = 2,
      main = "ROC Curve - Multiclass Classification (One-vs-All)")
plot(perf2, add = TRUE, col = "blue", lwd = 2)
plot(perf3, add = TRUE, col = "darkgreen", lwd = 2)

# Add the random-classifier diagonal
abline(a = 0, b = 1, lty = 2, col = "gray")

legend("bottomright",
      legend = c("Galaxy", "Quasar", "Star"),
      col = c("red", "blue", "darkgreen"),
      lwd = 2)

```



```
#AUC 'GALAXY'  
print(round(auc1, 4))
```

```
## [1] 0.9029
```

```
#AUC 'QSO'  
print(round(auc2, 4))
```

```
## [1] 0.9552
```

```
#AUC 'STAR'  
print(round(auc3, 4))
```

```
## [1] 0.8972
```

The graphical representation of the ROC curves seems to indicate good data classification. In theory, the curves should be as close as possible to the upper-left corner of the Cartesian plane, and as can be seen, the three curves considered are very close to this condition. The classification of GALAXY is particularly good among the three curves, which may have been influenced by the fact that this class is the most represented of the three, generating an imbalance in the classification. Considering all the results for this method, we are satisfied with the precision of the results and consider it a valid option.

4.2 Logistic Regression

```
#Training set
# remove outliers
# Create a modified version of the data without altering the
original
dati_mod <- dati
data_normalized2_mod <- data_normalized2

set.seed(123)
rows_to_keep <- !rownames(dati_mod) %in% rownames(to_remove)
dati_mod <- dati_mod[rows_to_keep, ]

numeric.training_mod <- as.matrix(dati_mod |> select(alpha,
delta, u, g, r, i, z, log_redshift, starts_with("cam_col")))
numeric.validation_mod <- as.matrix(data_normalized2_mod |>
select(alpha, delta, u, g, r, i, z, log_redshift,
starts_with("cam_col")))

class.training_mod <- dati_mod$class
class.validation_mod <- data_normalized2_mod$class

# Create the multinomial logistic regression model
# Creation of the multinomial logistic regression model
library(glmnet)
model <- glmnet(numeric.training_mod, class.training_mod, family
= 'multinomial', lambda = 0)

pred_class_mod <- predict(model, numeric.validation_mod, type =
"class")
pred_class_mod <- as.factor(pred_class_mod)
pred_probs <- predict(model, numeric.validation_mod, type =
"response")

pred_class_mod <- as.factor(pred_class_mod)
class.validation_mod <- as.factor(class.validation_mod)

pred_class_mod <- factor(pred_class_mod, levels =
levels(class.validation_mod))
conf_matrix <- confusionMatrix(pred_class_mod,
class.validation_mod)
print(conf_matrix)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction GALAXY   QSO   STAR
##   GALAXY   11802   480    1
##   QSO       181  3950    0
##   STAR       463    2  4834
##
## Overall Statistics
##
##              Accuracy : 0.9481
##              95% CI : (0.9451, 0.951)
##   No Information Rate : 0.5732
##   P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.9109
##
##   Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##              Class: GALAXY Class: QSO Class: STAR
## Sensitivity              0.9483      0.8912      0.9998
## Specificity              0.9481      0.9895      0.9724
## Pos Pred Value           0.9608      0.9562      0.9122
## Neg Pred Value           0.9317      0.9726      0.9999
## Prevalence               0.5732      0.2041      0.2227
## Detection Rate           0.5435      0.1819      0.2226
## Detection Prevalence     0.5657      0.1903      0.2440
## Balanced Accuracy        0.9482      0.9404      0.9861
```

```
#F1SCORE
f1 <- F1_Score(y_true = class.validation, y_pred =
pred_class_mod);f1
```

```
## [1] 0.9545069
```

```
# ROC CURVE AND AUC
```

```
pred_probs <- pred_probs[, , 1]
```

```
class_names_mod <- levels(class.validation_mod)
```

```
n_classes <- length(class_names_mod)
```

```
colors <- c("red", "blue", "green")
```

```
pred_roclogit <- prediction(pred_probs[, 1],
```

```
ifelse(class.validation_mod == class_names_mod[1], 1, 0))
```

```
perf_logit <- performance(pred_roclogit, "tpr", "fpr")
```

```
plot(perf_logit, col = colors[1], lwd = 2, main = "ROC Curve  
(One-vs-All)", xlim = c(0, 1), ylim = c(0, 1))
```

```
for (class_index in 2:n_classes) {
```

```
  class_binary <- ifelse(class.validation_mod ==  
class_names_mod[class_index], 1, 0)
```

```
  pred_roclogit <- prediction(pred_probs[, class_index],  
class_binary)
```

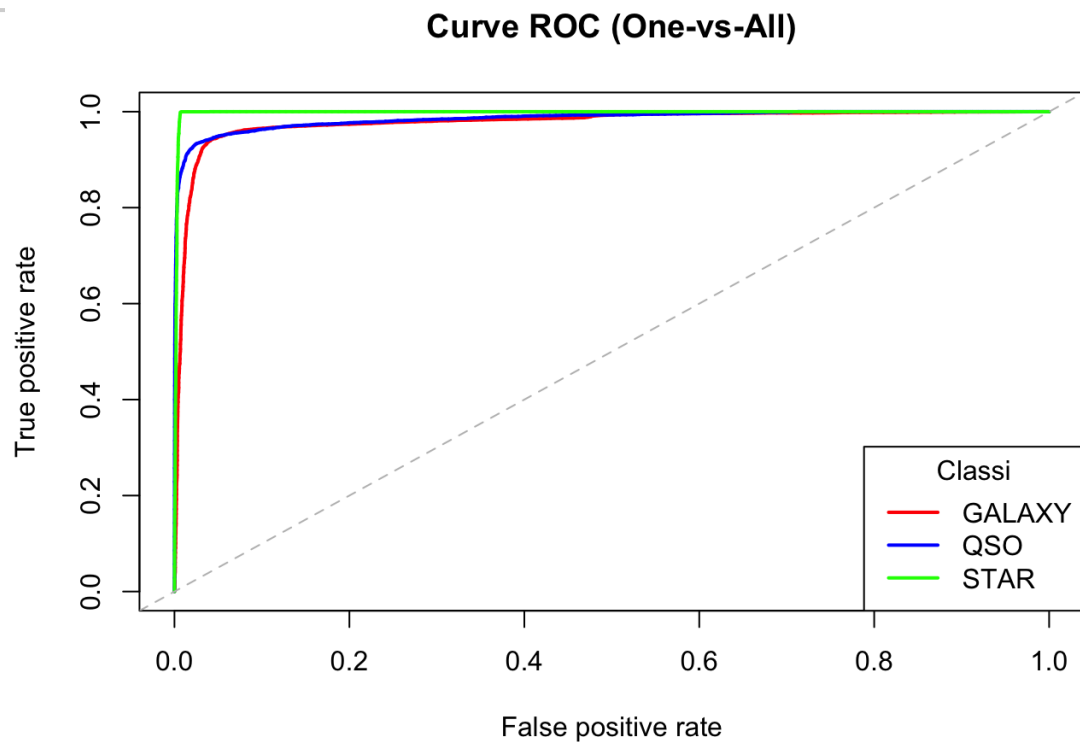
```
  perf_logit <- performance(pred_roclogit, "tpr", "fpr")
```

```
  plot(perf_logit, col = colors[class_index], lwd = 2, add =  
TRUE)
```

```
}
```

```
abline(a = 0, b = 1, lty = 2, col = "gray")
```

```
legend("bottomright", legend = class_names_mod, col = colors, lty  
= 1, lwd = 2, title = "Classes")
```



```
#AUC 'GALAXY'  
class_binary1 <- ifelse(class.validation_mod ==  
class_names_mod[1], 1, 0)  
pred_roclogit1 <- prediction(pred_probs[, 1], class_binary1)  
auc_logit1 <- performance(pred_roclogit1, "auc")  
auc_logit1@y.values[[1]]
```

```
## [1] 0.9776634
```

```
#AUC 'QSO'  
class_binary2 <- ifelse(class.validation_mod ==  
class_names_mod[2], 1, 0)  
pred_roclogit2 <- prediction(pred_probs[, 2], class_binary2)  
auc_logit2 <- performance(pred_roclogit2, "auc")  
auc_logit2@y.values[[1]]
```

```
## [1] 0.9854502
```

```
auc_logit1
```

```
## A performance instance  
## 'Area under the ROC curve'
```

```
#AUC STAR
class_binary3 <- ifelse(class.validation_mod ==
class_names_mod[3], 1, 0)
pred_roclogit3 <- prediction(pred_probs[, 3], class_binary3)
auc_logit3 <- performance(pred_roclogit3, "auc")
auc_logit3@y.values[[1]]
```

```
## [1] 0.9980794
```

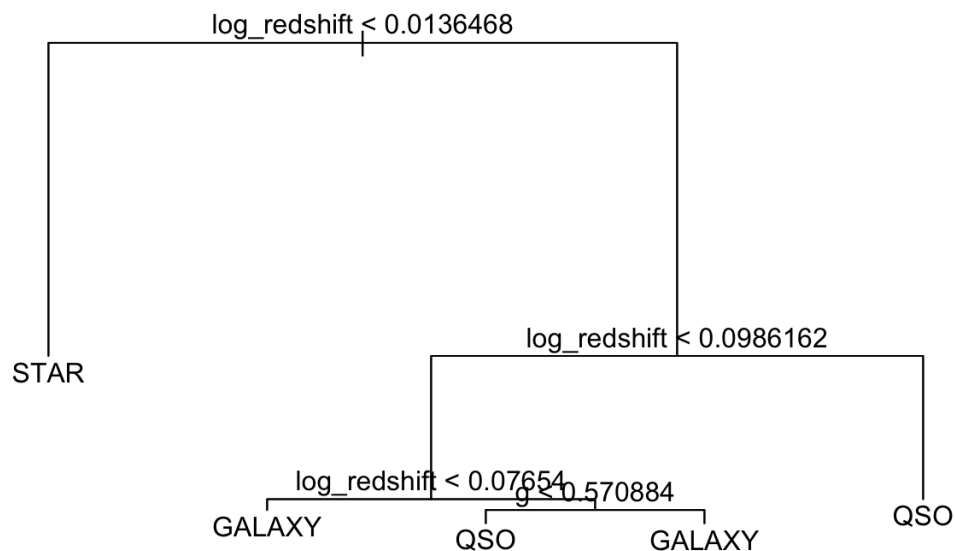
For logistic regression, we used the `glmnet` package to implement a multiclass regression model. The model has no penalization; in fact, λ is equal to zero, allowing it to fit the training data without restrictions on model complexity. To evaluate model performance and, specifically, given the lack of penalization, its ability not to overfit the data, we evaluated it on the validation set. We calculated accuracy and the F1-score and, using the one-vs-all approach, implemented graphical and numerical metrics such as the ROC curve and AUC to evaluate performance. All metrics were more than satisfactory, showing overall accuracy above 94% and class-specific sensitivity and specificity values above 85% for all three classes. In this context, the STAR class stood out with a sensitivity of 99.98%, and QSO with a specificity of 99.02%. The ROC curve shows strong discriminant ability of the classifier, adhering to the upper-left corner for all three classes. This interpretation is supported by the AUC values, which tend toward 1 and again indicate the model's strong ability to separate the classes. Finally, the F1-score is 95%, once again indicating good classification ability even in the presence of class imbalance. The non-penalized multiclass logistic regression is therefore considered the best classification model among those proposed, as it does not show evident signs of overfitting. It should be noted, however, that penalization could be appropriate in contexts with greater complexity or a larger number of variables in order to ensure good generalization.

4.3 Decision Trees

```
library(tree)
tree.stars <- tree(class ~ ., data = dati, method = "class")
summary(tree.stars)
```

```
##
## Classification tree:
## tree(formula = class ~ ., data = dati, method = "class")
## Variables actually used in tree construction:
## [1] "log_redshift" "g"
## Number of terminal nodes: 5
## Residual mean deviance: 0.3507 = 9853 / 28100
## Misclassification error rate: 0.04605 = 1294 / 28102
```

```
# The error rate is 4%
plot(tree.stars)
text(tree.stars, pretty = 0)
```



```
train.pred <- predict(tree.stars , dati , type = "class")
table(train.pred , dati$class)
```

```
##
## train.pred GALAXY   QSO   STAR
##   GALAXY   11464   851    4
##   QSO       332  6946    0
##   STAR      106    1  8398
```

```
# Calculate the percentage of correctly classified observations
in the training set
(11464+6946+8398)/nrow(dati)
```

```
## [1] 0.9539535
```

```
tree.pred <- predict(tree.stars , data_normalized2 , type =
"class")
table(tree.pred, data_normalized2$class)
```



```
##
## tree.pred GALAXY   QSO  STAR
##   GALAXY  11477   420    1
##   QSO      644  4011    0
##   STAR     325    1  4834
```

```
# Calculate the percentage of correctly classified observations
in the test set
(11477+4011+4834)/nrow(data_normalized2)
```

```
## [1] 0.935937
```

```
# 92% of the observations are correctly classified with the
decision-tree method
```

```
#F1SCORE
f1 <- F1_Score(y_true = class.validation, y_pred = tree.pred) ;
f1
```

```
## [1] 0.9429017
```

ROC CURVE AND AUC

```
numeric.training <- as.data.frame(data1 |> select(alpha, delta, u,
g, r, i, z, log_redshift, cam_col))
numeric.validation <- as.data.frame(data_normalized2 |>
select(alpha, delta, u, g, r, i, z, log_redshift, cam_col))
# To extract probabilities, we convert to a data frame; otherwise
it is not possible
tree.prob <- predict(tree.stars, numeric.validation, type =
"vector")
```

"Galaxy" vs "non-Galaxy"

```
tree_pred1 <- predict(tree.stars, numeric.validation, type =
"class")
prob_tree1 <- ifelse(tree_pred1 == "GALAXY", 1, 0)
```

```
pred_tree1 <- prediction(prob_tree1, valid_bin1)
perf_tree1 <- performance(pred_tree1, "tpr", "fpr")
auc_tree1 <- performance(pred_tree1, "auc")@y.values[[1]]
```

"Quasar" vs "non-Quasar"

```
tree_pred2 <- predict(tree.stars, numeric.validation, type =
"class")
prob_tree2 <- ifelse(tree_pred2 == "QS0", 1, 0)
```

```
pred_tree2 <- prediction(prob_tree2, valid_bin2)
perf_tree2 <- performance(pred_tree2, "tpr", "fpr")
auc_tree2 <- performance(pred_tree2, "auc")@y.values[[1]]
```

"Star" vs "Non-Stars"

```
tree_pred3 <- predict(tree.stars, numeric.validation, type =
"class")
prob_tree3 <- ifelse(tree_pred3 == "STAR", 1, 0) # Probability
"1" (Star)
```

```
pred_tree3 <- prediction(prob_tree3, valid_bin3)
perf_tree3 <- performance(pred_tree3, "tpr", "fpr")
auc_tree3 <- performance(pred_tree3, "auc")@y.values[[1]]
```

Graphical representation of the ROC curve for the three classes

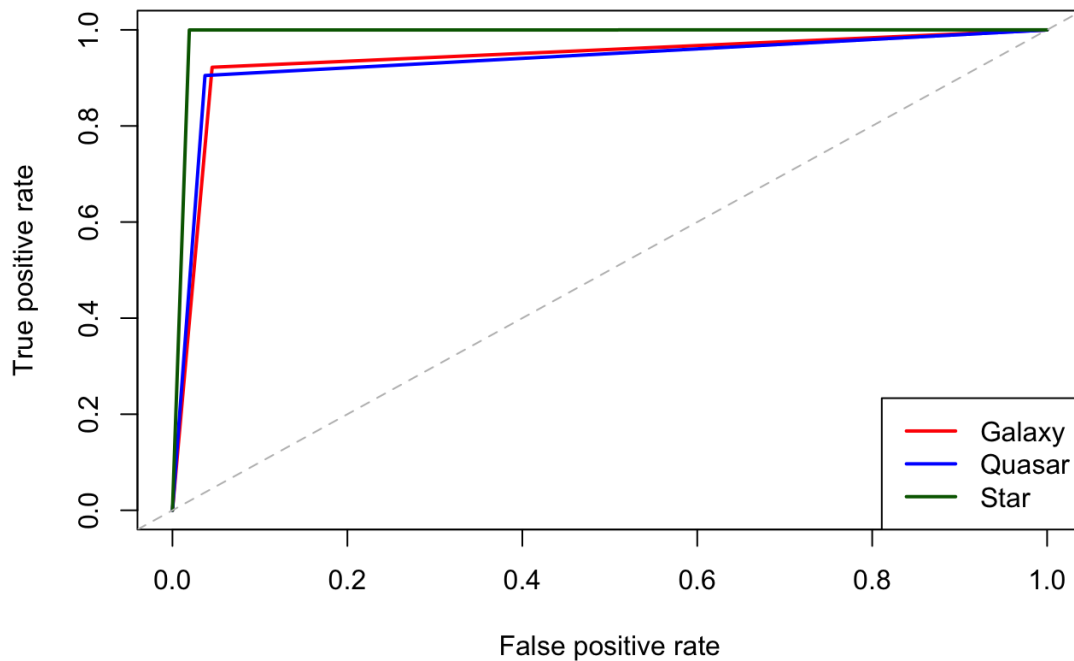
```
plot(perf_tree1, col = "red", lwd = 2,
      main = "ROC Curve - Multiclass Classification (One-vs-All)")
plot(perf_tree2, add = TRUE, col = "blue", lwd = 2)
plot(perf_tree3, add = TRUE, col = "darkgreen", lwd = 2)
```

```
abline(a = 0, b = 1, lty = 2, col = "gray")
```

```
legend("bottomright",
```

```
legend = c("Galaxy", "Quasar", "Star"),  
col = c("red", "blue", "darkgreen"),  
lwd = 2)
```

ROC Curve - Classificazione Multiclasse (One-vs-All)



```
#AUC 'GALAXY'  
print(round(auc_tree1, 4))
```

```
## [1] 0.9384
```

```
#AUC 'QS0'  
print(round(auc_tree2, 4))
```

```
## [1] 0.9339
```

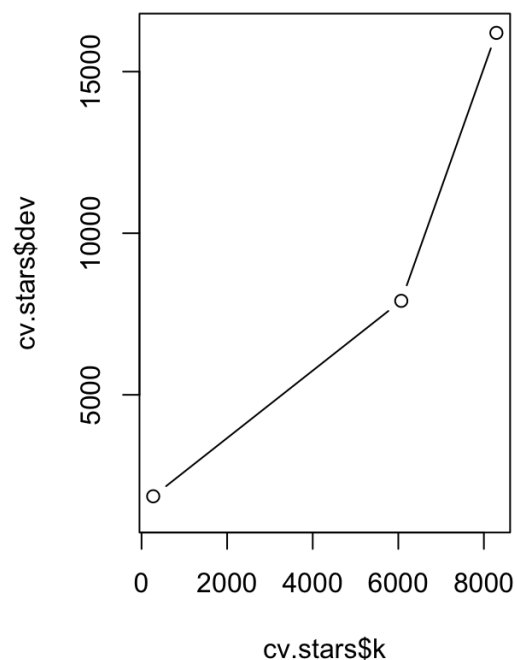
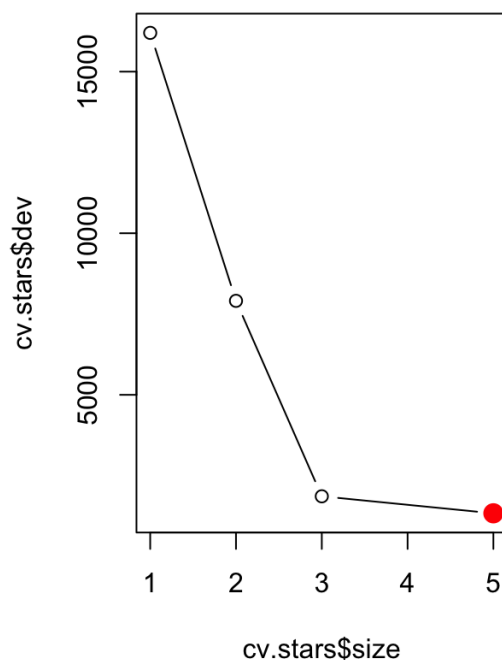
```
#AUC 'STAR'  
print(round(auc_tree3, 4))
```

```
## [1] 0.9902
```

```
# Apply pruning to see whether the algorithm could be optimized  
  
cv.stars <- cv.tree(tree.stars , FUN = prune.misclass) ; cv.stars
```

```
## $size
## [1] 5 3 2 1
##
## $dev
## [1] 1334 1860 7908 16200
##
## $k
## [1] -Inf 272.5 6069.0 8292.0
##
## $method
## [1] "misclass"
##
## attr("class")
## [1] "prune" "tree.sequence"
```

```
par(mfrow = c(1, 2))
plot(cv.stars$size , cv.stars$dev, type = "b")
points(cv.stars$size[which.min(cv.stars$dev)],
cv.stars$dev[which.min(cv.stars$dev)], col = "red", pch = 19, cex
= 1.5)
plot(cv.stars$k, cv.stars$dev, type = "b")
```



```
par(mfrow = c(1, 1))
```

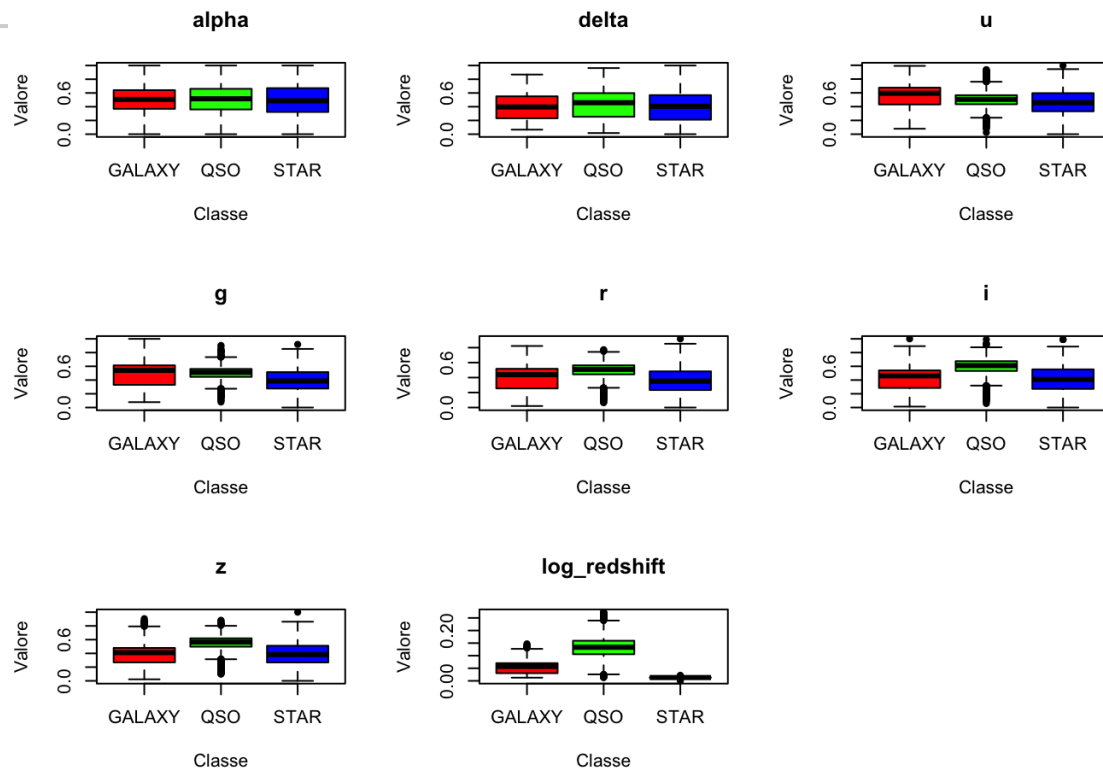
```
# Choose k = 5 nodes and apply the command to prune the tree
#prune.stars <- prune.misclass(tree.stars , best = 5)
#Error in prune.tree(tree = cv.stars, best = 5, method =
"misclass") : not legitimate tree
# Our tree cannot be pruned further.
```

4.4 LDA - Linear Discriminant Analysis

Implementation of generative models such as QDA and LDA, which belong to discriminant analysis. These models differ from discriminative models because they consider the distribution of the variables separately for each class. Before applying the models, it is necessary to verify that their assumptions are satisfied.

```
# 1. Check common variance across classes using class-conditional
boxplots
par(mfrow = c(3,3))
balanced_numeric_vars <- dati[sapply(dati, is.numeric)]

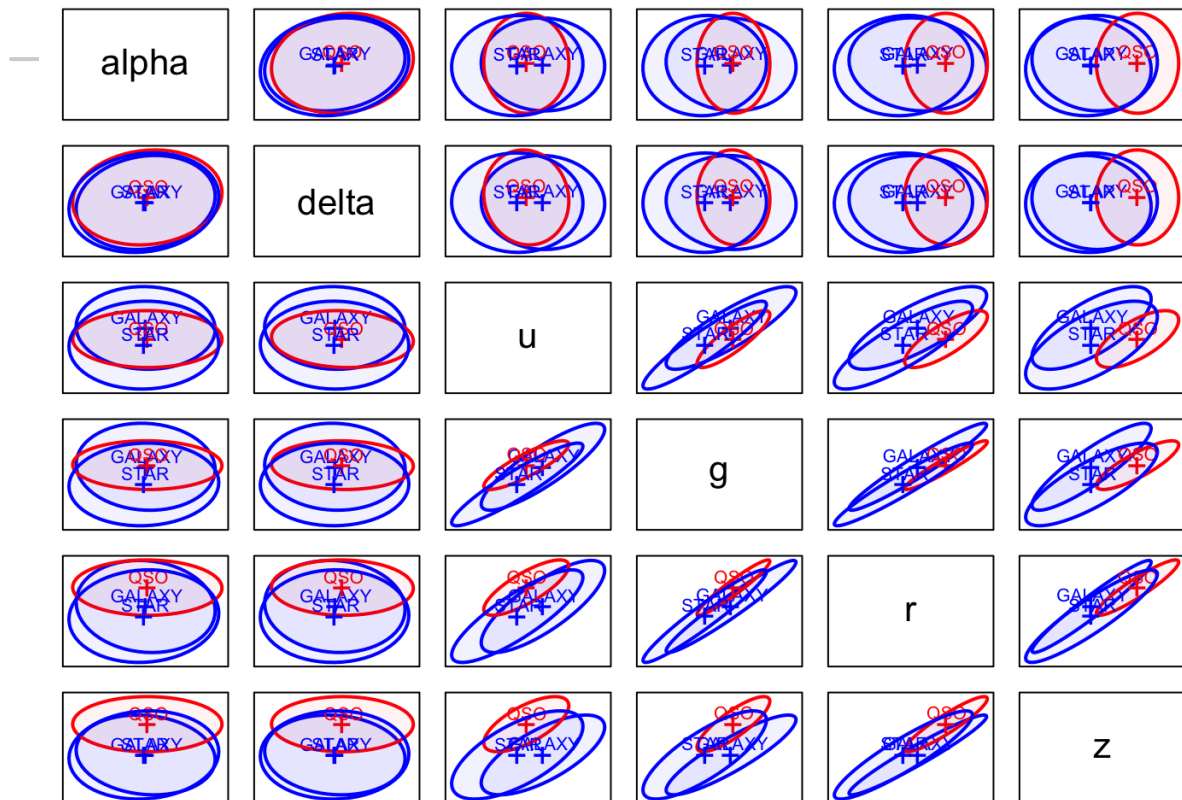
for (var in colnames(balanced_numeric_vars)) {
  boxplot(balanced_numeric_vars[[var]] ~ dati$class, # Use
data_clean$class for consistency
        main = var,
        xlab = "Class",
        ylab = "Value",
        col = c("red", "green", "blue"),
        outline = TRUE,
        pch = 19)
}
par(mfrow = c(1,1))
```



From the boxplots, it can be observed that for some variables the variance is similar across classes, while for others, such as log - redshift or the variables z and g , clear differences are visible. This suggests that the common variance assumption may not be satisfied for all variables.

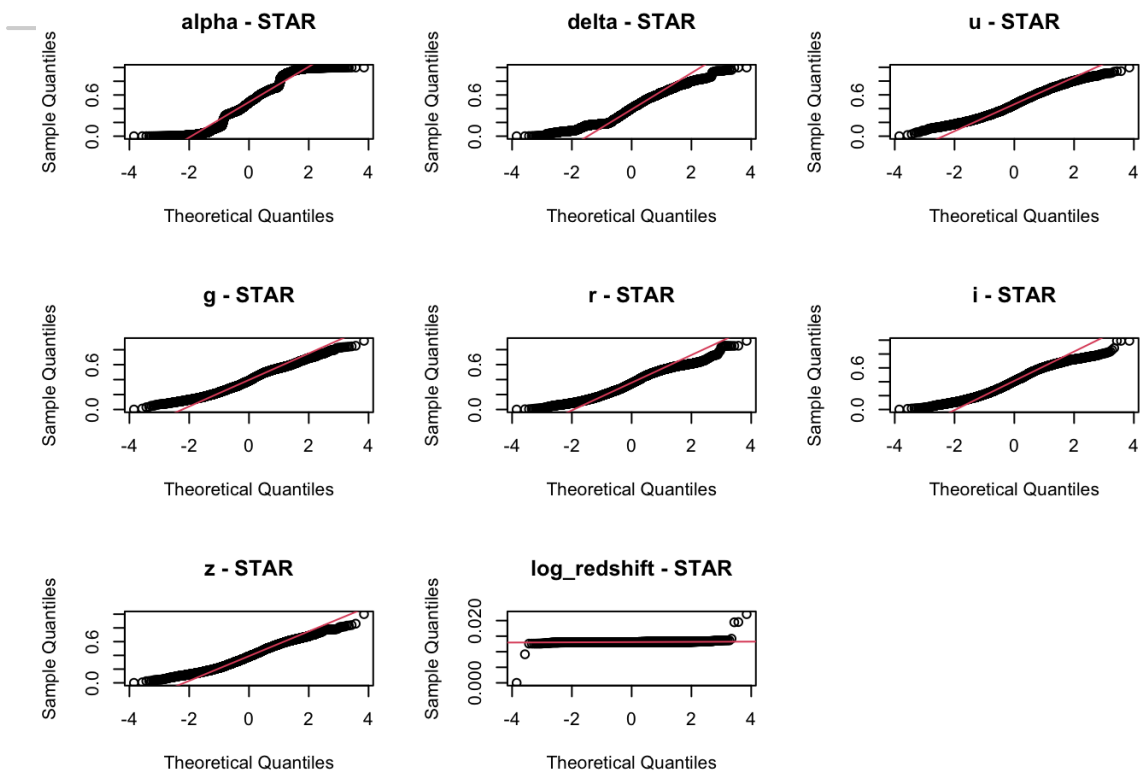
```
# 2. Check covariance among variables
# Graphically analyze whether the covariance matrix can be
# considered common across classes.
#install.packages("candisc")
library(candisc)

covEllipses(balanced_numeric_vars[,1:(dim(balanced_numeric_vars)
[2]-1)],
            factor(dati$class),
            fill = TRUE,
            pooled = FALSE,
            col = c("blue", "red"),
            variables = c(1:5, (dim(balanced_numeric_vars)
[2]-1)),
            fill.alpha = 0.05)
```

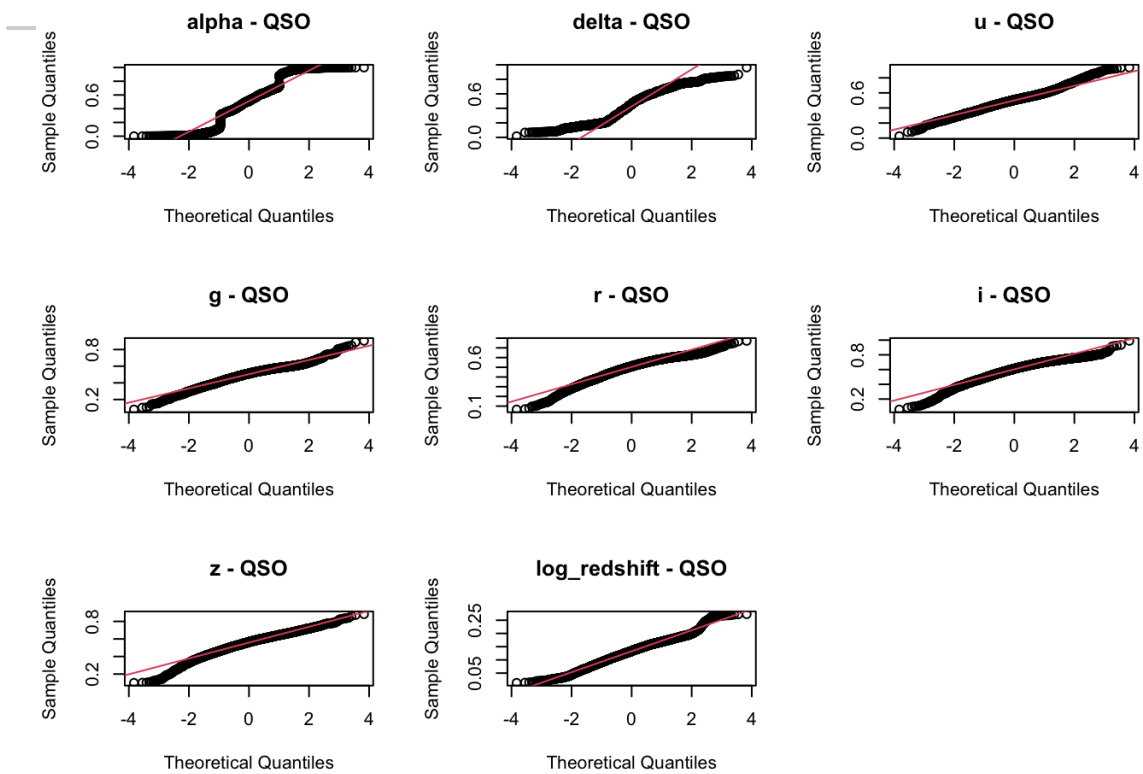


The graphical analysis of the covariance ellipses suggests that, for some variables, the covariance matrix across classes is similar, supporting the common covariance assumption. However, for other variables (e.g. r and z), significant differences in the size and orientation of the ellipses are observed, indicating that the assumption may not be valid for all features. This could affect the effectiveness of LDA, which assumes common covariance across classes. We will evaluate and compare the result obtained from this model with QDA, which does not require this type of assumption.

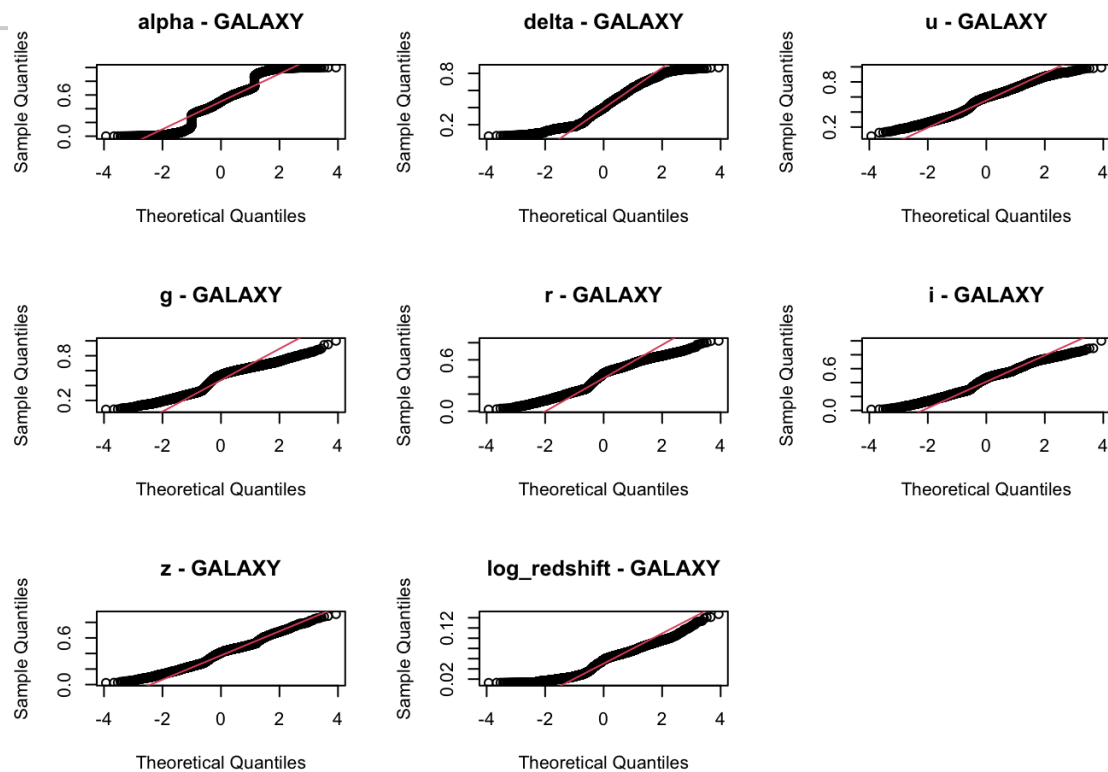
```
# Check class-conditional normality using QQ-plots
# Construction of QQ-plots to check normality of class-
conditional observations (the first four numeric variables are
analyzed)
par(mfrow = c(3, 3))
for (i in colnames(balanced_numeric_vars)) {
  qqnorm(dati[dati$class == "STAR", ][[i]], main = paste(i, "-
STAR"))
  qqline(dati[dati$class == "STAR", ][[i]], col = 2)
}
par(mfrow = c(3, 3))
```



```
for (i in colnames(balanced_numeric_vars)) {
  qqnorm(dati[dati$class == "QSO", ][[i]], main = paste(i, "-
QSO"))
  qqline(dati[dati$class == "QSO", ][[i]], col = 2)
}
par(mfrow = c(3, 3))
```

```
for (i in colnames(balanced_numeric_vars)) {
  qqnorm(dati[dati$class == "GALAXY", ][[i]], main = paste(i, "-
GALAXY"))
  qqline(dati[dati$class == "GALAXY", ][[i]], col = 2)
}
par(mfrow = c(1, 1))
```

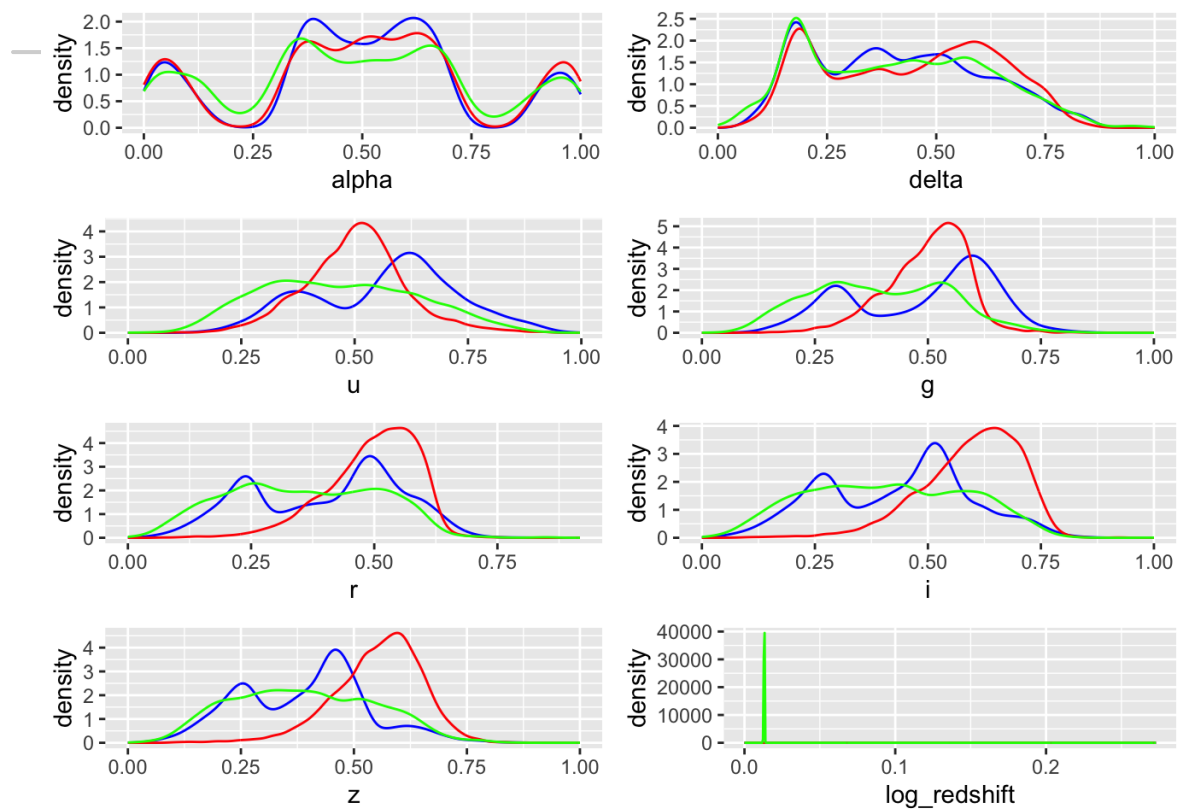


Analyzing the QQ-plots of the three classes, the variables do not appear to be normally distributed. They show significant deviations in the tails from the bisector used as the reference for assessing normality, and some variables, such as alpha, also clearly deviate from the theoretical normal distribution in the central part of the distribution. With respect to this third assumption, as with the previous ones, we are not inclined to state that it is satisfied by the data, and we would therefore prefer not to use the LDA model for this reason.

```
# Check the distribution using the density function
library(ggplot2)
library(gridExtra)

plot_density <- list()
# Construction of density curves conditional on class
for(i in colnames(balanced_numeric_vars)){
  plot_density[[i]] <- ggplot(dati, aes_string(x = i, y =
"..density..", col = "class")) +
    geom_density(aes(y = ..density..)) +
    scale_color_manual(values = c("blue", "red", "green")) + #
  Add a third color
    theme(legend.position = "none")
}

do.call(grid.arrange, c(plot_density, nrow = 4))
```



The graphical representation shows that the variables do not satisfy the condition of class-conditional normality. Graphically, the variable distributions appear mostly bimodal and non-symmetric, and we are not inclined to conclude that normality is present.

```

# Shapiro-Wilk test
numeric_vars <- colnames(balanced_numeric_vars)
[sapply(balanced_numeric_vars, is.numeric)]

pvalue_shapiro <- matrix(0, nrow = length(numeric_vars), ncol =
3)
rownames(pvalue_shapiro) <- numeric_vars
colnames(pvalue_shapiro) <- c("QSO", "STAR", "GALAXY")

# Run the Shapiro test for each numeric variable, conditional on
class
#for (i in colnames(balanced_numeric_vars)
[sapply(balanced_numeric_vars, is.numeric)]) {
  #pvalue_shapiro[i, "STAR"] <-
shapiro.test(balanced_numeric_vars[dati$class == "STAR", ][[i]])
$p.value
  #pvalue_shapiro[i, "QSO"] <-
shapiro.test(balanced_numeric_vars[dati$class == "QSO", ][[i]])
$p.value
  #pvalue_shapiro[i, "GALAXY"] <-
shapiro.test(balanced_numeric_vars[dati$class == "GALAXY", ]
[[i]])$p.value
#}

#COMMENT: Error in shapiro.test(balanced_numeric_vars[dati$class
== "STAR", ][[i]]) :
#sample size must be between 3 and 5000 : our dataset has too
many observations, so we cannot apply the test

```

```

library(MASS)
library(caret)

lda.fit <- lda(class ~ ., data = dati)
print(lda.fit)

```

```
## Call:
## lda(class ~ ., data = dati)
##
## Prior probabilities of groups:
##      GALAXY      QSO      STAR
## 0.4235286 0.2774891 0.2989823
##
## Group means:
##      alpha      delta      u      g
##      i      z  cam_col2  cam_col3  cam_col4  cam_col5
##      cam_col6 log_redshift
## GALAXY 0.4969442 0.4095841 0.5674997 0.4903636 0.4022899
##         0.4324575 0.3901114 0.1675349 0.1903882 0.2009746 0.1908083
##         0.1131743 0.05337641
## QSO     0.5062428 0.4405584 0.5021376 0.4991554 0.4957660
##         0.5951550 0.5535025 0.1750449 0.2067197 0.1881252 0.1750449
##         0.1320852 0.13114279
## STAR    0.4870138 0.4083889 0.4667675 0.3934313 0.3545329
##         0.4115396 0.3899495 0.1848369 0.1667460 0.2004285 0.1686503
##         0.1493692 0.01310459
##
## Coefficients of linear discriminants:
##      LD1      LD2
## alpha      0.10541483 -0.02740865
## delta      0.26962094 0.16619071
## u          -2.39169384 0.88038221
## g          -2.58969167 -19.10529520
## r          -1.82794161 8.99152073
## i          4.67247190 6.65614163
## z          -0.33977518 2.31319512
## cam_col2    0.04967659 0.14562052
## cam_col3    0.11675637 0.02232845
## cam_col4    0.08717260 0.07062595
## cam_col5    0.01838221 0.05055021
## cam_col6    0.04956033 0.24819550
## log_redshift 39.86178791 -12.60272132
##
## Proportion of trace:
##      LD1      LD2
## 0.8755 0.1245
```

```
class.validation <- factor(data_normalized2$class)

numeric.validation <- as.data.frame(numeric.validation) %>%
  mutate(
    alpha = as.numeric(alpha),
    delta = as.numeric(delta),
    u = as.numeric(u),
    g = as.numeric(g),
    r = as.numeric(r),
    i = as.numeric(i),
    z = as.numeric(z),
    log_redshift = as.numeric(log_redshift),
    cam_col = as.factor(cam_col)
  )

lda_pred <- predict(lda.fit, numeric.validation)
lda_class <- factor(lda_pred$class, levels =
  levels(class.validation))

confusionMatrix(lda_class, class.validation) # Accuracy :
0.7852
```

Confusion Matrix and Statistics

##

Reference

Prediction GALAXY QSO STAR

GALAXY 8367 235 162

QSO 518 4010 0

STAR 3561 187 4673

##

Overall Statistics

##

Accuracy : 0.7852

95% CI : (0.7797, 0.7907)

No Information Rate : 0.5732

P-Value [Acc > NIR] : < 2.2e-16

##

Kappa : 0.6643

##

McNemar's Test P-Value : < 2.2e-16

##

Statistics by Class:

##

Class: GALAXY Class: QSO Class: STAR

Sensitivity 0.6723 0.9048 0.9665

Specificity 0.9572 0.9700 0.7779

Pos Pred Value 0.9547 0.8856 0.5549

Neg Pred Value 0.6850 0.9754 0.9878

Prevalence 0.5732 0.2041 0.2227

Detection Rate 0.3853 0.1847 0.2152

Detection Prevalence 0.4036 0.2085 0.3878

Balanced Accuracy 0.8147 0.9374 0.8722

#F1SCORE

```
f1 <- F1_Score(y_true = class.validation, y_pred = lda_class);
```

```
f1
```

```
## [1] 0.7889675
```

```

# ROC CURVE AND AUC
# "Galaxy" vs "non-Galaxy"
lda_pred1 <- predict(lda.fit, numeric.validation)
lda_class1 <- factor(lda_pred1$class, levels =
levels(class.validation))
prob_lda1 <- ifelse(lda_class1 == "GALAXY", 1, 0)

pred_lda1 <- prediction(prob_lda1, valid_bin1)
perf_lda1 <- performance(pred_lda1, "tpr", "fpr")
auc_lda1 <- performance(pred_lda1, "auc")@y.values[[1]]

# "Quasar" vs "non-Quasar"
lda_pred2 <- predict(lda.fit, numeric.validation)
lda_class2 <- factor(lda_pred2$class, levels =
levels(class.validation))
prob_lda2 <- ifelse(lda_class2 == "QSO", 1, 0)

pred_lda2 <- prediction(prob_lda2, valid_bin2)
perf_lda2 <- performance(pred_lda2, "tpr", "fpr")
auc_lda2 <- performance(pred_lda2, "auc")@y.values[[1]]

# "Star" vs "Non-Stars"
lda_pred3 <- predict(lda.fit, numeric.validation)
lda_class3 <- factor(lda_pred3$class, levels =
levels(class.validation))
prob_lda3 <- ifelse(lda_class3 == "STAR", 1, 0) # Probability
"1" (Star)

pred_lda3 <- prediction(prob_lda3, valid_bin3)
perf_lda3 <- performance(pred_lda3, "tpr", "fpr")
auc_lda3 <- performance(pred_lda3, "auc")@y.values[[1]]

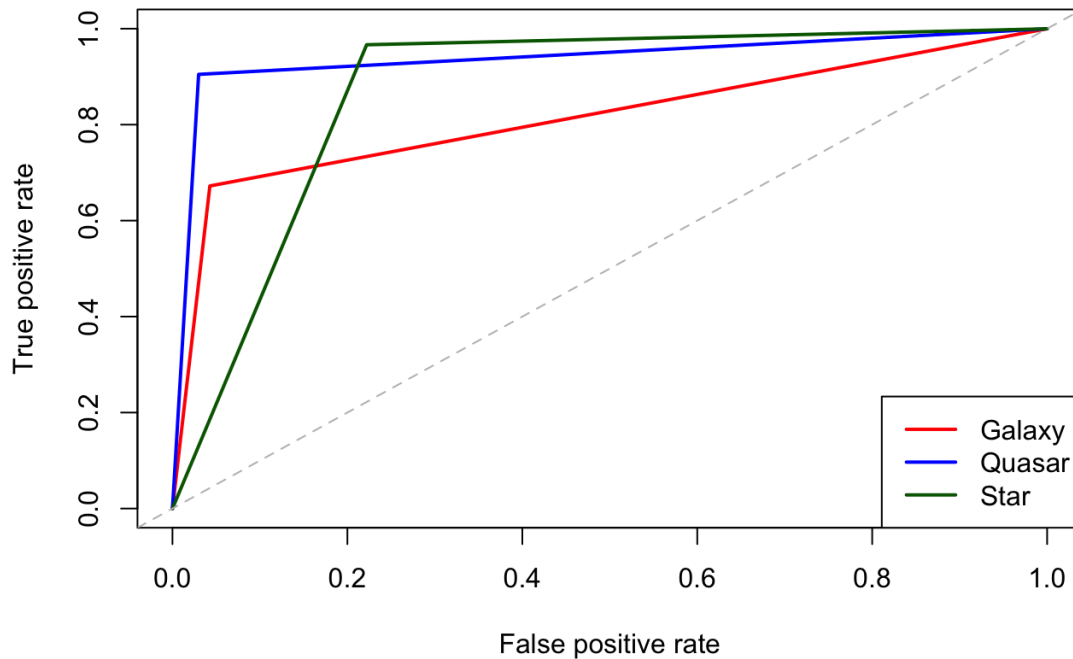
### Graphical representation of the ROC curve for the three
classes
plot(perf_lda1, col = "red", lwd = 2,
      main = "ROC Curve - Multiclass Classification (One-vs-All)")
plot(perf_lda2, add = TRUE, col = "blue", lwd = 2)
plot(perf_lda3, add = TRUE, col = "darkgreen", lwd = 2)

abline(a = 0, b = 1, lty = 2, col = "gray")

legend("bottomright",
      legend = c("Galaxy", "Quasar", "Star"),
      col = c("red", "blue", "darkgreen"),
      lwd = 2)

```


ROC Curve - Classificazione Multiclasse (One-vs-All)



```
#AUC 'GALAXY'  
print(round(auc_lda1, 4))
```

```
## [1] 0.8147
```

```
#AUC 'QSO'  
print(round(auc_lda2, 4))
```

```
## [1] 0.9374
```

```
#AUC 'STAR'  
print(round(auc_lda3, 4))
```

```
## [1] 0.8722
```

Since the initial assumptions of normal variable distribution and homoscedasticity of variance were not verified, we conclude that the LDA model is not appropriate for classifying our data. The classification evaluation results obtained may not be valid, because the assumptions of this method are not guaranteed for the data to which it is applied. This model also causes underfitting, since the bias is very high and does not lead to an optimal evaluation. We proceed by applying QDA to our data to assess its effectiveness, since it does not require the conditions that are instead necessary for the linear model.

4.5 QDA - Quadratic Discriminant Analysis

```
qda_model <- qda(class ~ ., data = dati)
qda_model
```

```
## Call:
## qda(class ~ ., data = dati)
##
## Prior probabilities of groups:
##      GALAXY      QS0      STAR
## 0.4235286 0.2774891 0.2989823
##
## Group means:
##           alpha      delta      u      g
r           i      z  cam_col2  cam_col3  cam_col4  cam_col5
cam_col6 log_redshift
## GALAXY 0.4969442 0.4095841 0.5674997 0.4903636 0.4022899
0.4324575 0.3901114 0.1675349 0.1903882 0.2009746 0.1908083
0.1131743 0.05337641
## QS0    0.5062428 0.4405584 0.5021376 0.4991554 0.4957660
0.5951550 0.5535025 0.1750449 0.2067197 0.1881252 0.1750449
0.1320852 0.13114279
## STAR   0.4870138 0.4083889 0.4667675 0.3934313 0.3545329
0.4115396 0.3899495 0.1848369 0.1667460 0.2004285 0.1686503
0.1493692 0.01310459
```

```
# The output shows the means for each classification group
```

```
# In the training set
qda.class2 <- predict(qda_model, dati)$class
table(qda.class2, dati$class)
```

```
##
## qda.class2 GALAXY  QS0  STAR
##      GALAXY 11120  501    9
##      QS0    668  7295    1
##      STAR   114    2  8392
```

```
# Accuracy in the training set:
(11120+7295+8392)/nrow(dati)
```

```
## [1] 0.9539179
```

```
# In the test set
qda.class <- predict(qda_model, data_normalized2)$class
table(qda.class, class.validation)
```

```
##           class.validation
## qda.class GALAXY    QSO  STAR
##   GALAXY  11899  1046  2482
##   QSO      378  3385  2348
##   STAR     169    1    5
```

```
# Accuracy in the test set:
(12145+3136+6)/nrow(data_normalized2)
```

```
## [1] 0.7040483
```

```
# 69.2% of the observations are correctly classified by QDA in
the test set
```

```
#F1SCORE
f1 <- F1_Score(y_true = class.validation, y_pred = tree.pred) ;
f1
```

```
## [1] 0.9429017
```

Comparing the confusion matrix results for the training set and the validation set, we observe a clear difference: in the training set, the QDA model appears to work quite well, while on the validation set the results are clearly disappointing, especially for the STAR class. This difference is explained by the nature of Quadratic Discriminant Analysis, which tends to overfit the data because it is a more flexible model than the linear one. For this reason, performance on the training data is almost optimal, whereas when the dataset changes and the model is applied to the validation set, the results are poor and the model is unable to generalize sufficiently.

```

# ROC CURVE AND AUC
# "Galaxy" vs "non-Galaxy"
qda_pred1 <- predict(qda_model, numeric.validation)$class
qda_class1 <- factor(qda_pred1, levels =
levels(class.validation))
prob_qda1 <- ifelse(qda_class1 == "GALAXY", 1, 0)

pred_qda1 <- prediction(prob_qda1, valid_bin1)
perf_qda1 <- performance(pred_qda1, "tpr", "fpr")
auc_qda1 <- performance(pred_qda1, "auc")@y.values[[1]]

# "Quasar" vs "non-Quasar"
qda_pred2 <- predict(qda_model, numeric.validation)$class
qda_class2 <- factor(qda_pred2, levels =
levels(class.validation))
prob_qda2 <- ifelse(qda_class2 == "QSO", 1, 0)

pred_qda2 <- prediction(prob_qda2, valid_bin2)
perf_qda2 <- performance(pred_qda2, "tpr", "fpr")
auc_qda2 <- performance(pred_qda2, "auc")@y.values[[1]]

# "Star" vs "Non-Stars"
qda_pred3 <- predict(qda_model, numeric.validation)$class
qda_class3 <- factor(qda_pred3, levels =
levels(class.validation))
prob_qda3 <- ifelse(qda_class3 == "STAR", 1, 0)

pred_qda3 <- prediction(prob_qda3, valid_bin3)
perf_qda3 <- performance(pred_qda3, "tpr", "fpr")
auc_qda3 <- performance(pred_qda3, "auc")@y.values[[1]]

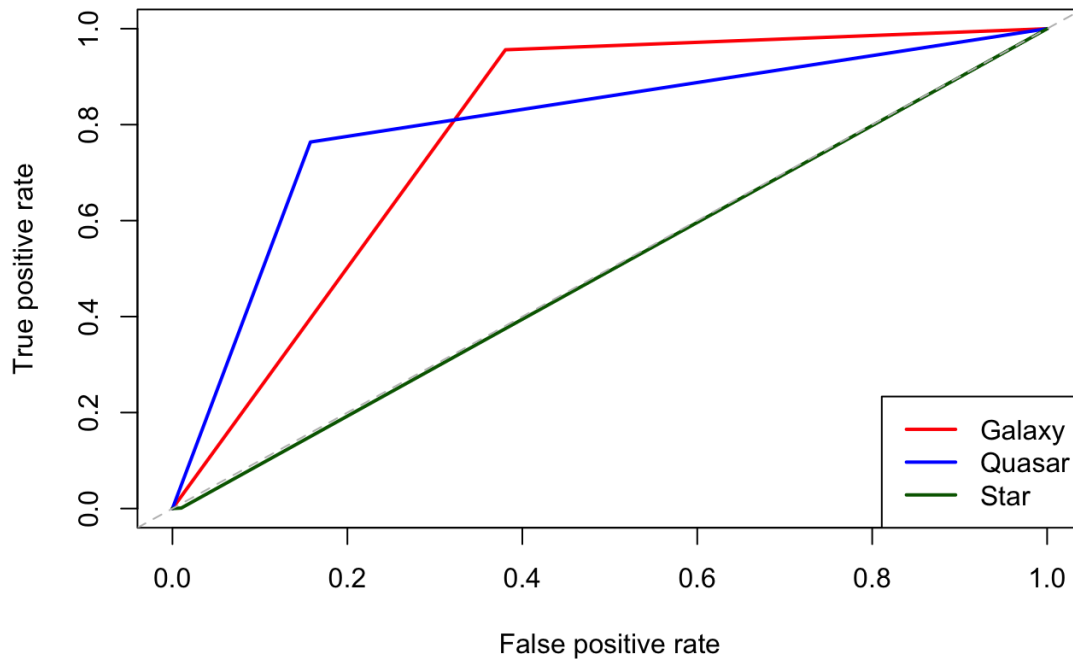
### Graphical representation of the ROC curve for the three
classes
plot(perf_qda1, col = "red", lwd = 2,
      main = "ROC Curve - Multiclass Classification (One-vs-All)")
plot(perf_qda2, add = TRUE, col = "blue", lwd = 2)
plot(perf_qda3, add = TRUE, col = "darkgreen", lwd = 2)

abline(a = 0, b = 1, lty = 2, col = "gray")

legend("bottomright",
      legend = c("Galaxy", "Quasar", "Star"),
      col = c("red", "blue", "darkgreen"),
      lwd = 2)

```

ROC Curve - Classificazione Multiclasse (One-vs-All)



```
#AUC 'GALAXY'  
print(round(auc_qda1, 4))
```

```
## [1] 0.7877
```

```
#AUC 'QSO'  
print(round(auc_qda2, 4))
```

```
## [1] 0.803
```

```
#AUC 'STAR'  
print(round(auc_qda3, 4))
```

```
## [1] 0.4955
```

The ROC curve results are not optimal for the classification of STAR. However, we should consider that what is shown is the result of converting a three-class problem into a binary problem. This change may have influenced the model output, since we had previously observed that the QDA model has some difficulty classifying the third category. This weakness is probably amplified when the GALAXY and QSO classes are combined, relative to which STAR appears to be classified very poorly, practically at the level of a random 50% classification.

5 Deployment

Based on the performance obtained in the previous phase, we choose to proceed with the logistic regression model, which showed the best discriminant ability among the classes. We consider as the training set the union of the previous training and validation sets, which had been used to train the classifier and find the best estimates for the hyperparameters. We now evaluate performance using the test set. Unlike the training and validation sets, which are subject to cleaning and preprocessing, the test set remains 'raw', presenting new data that the models were not trained to handle, thereby assessing their generalization ability.

```
test$log_redshift <- log(test$redshift + 1)
test <- test[,-10]

train <- rbind(dati, data_normalized2)

set.seed(123)

# Prepare the data
numeric.train <- as.matrix(train |> select(alpha, delta, u, g, r,
i, z, log_redshift, cam_col))
numeric.test <- as.matrix(test |> select(alpha, delta, u, g, r,
i, z, log_redshift, cam_col))

class.train <- factor(train$class)
class.test <- factor(test$class)

# Multinomial logistic regression model
mod <- glmnet(numeric.train, class.train, family = 'multinomial',
lambda = 0)

# Predictions
pred_class_test <- predict(mod, numeric.test, type = "class")
pred_class_test <- factor(pred_class_test, levels =
levels(class.train))
pred_probs_test <- predict(mod, numeric.test, type = "response")

# Confusion matrix
conf_matrix <- confusionMatrix(pred_class_test, class.test)
print(conf_matrix)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction GALAXY  QS0  STAR
##    GALAXY    7808 1886   42
##    QS0        175 1625   37
##    STAR       3426  127 3892
##
## Overall Statistics
##
##              Accuracy : 0.7007
##              95% CI : (0.6941, 0.7072)
##    No Information Rate : 0.5999
##    P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.4949
##
##    Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##              Class: GALAXY Class: QS0 Class: STAR
## Sensitivity                0.6844    0.44667    0.9801
## Specificity                0.7466    0.98622    0.7639
## Pos Pred Value             0.8020    0.88459    0.5228
## Neg Pred Value             0.6120    0.88284    0.9932
## Prevalence                 0.5999    0.19129    0.2088
## Detection Rate             0.4106    0.08545    0.2046
## Detection Prevalence       0.5119    0.09659    0.3915
## Balanced Accuracy          0.7155    0.71644    0.8720
```

```
# F1 Score
f1 <- F1_Score(y_true = class.test, y_pred = pred_class_test)
f1
```

```
## [1] 0.7385197
```

Compared with the results obtained on the validation set, the model shows less satisfactory but still positive results. As explained above, the lack of penalization of model complexity harms its generalization ability, favoring a tendency to overfit the training set and therefore producing a lower mean squared error than on the test set. In the confusion matrix, the most critical value is the true positive rate of 44%; the other true positive rates are above 68%. The STAR class has a sensitivity value of 98%. The F1-score is 73%, suggesting that the model can predict the classes relatively reliably.